

Search Problems

Artificial Intelligence

Giorgio Fumera

giorgio.fumera@unica.it

Department of Electrical and Electronic Engineering



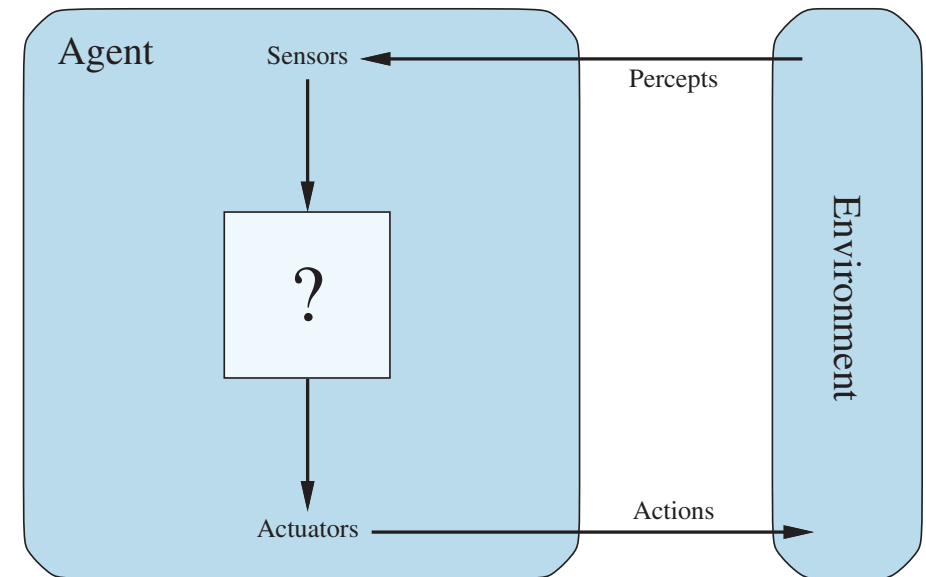
UNICA

UNIVERSITÀ
DEGLI STUDI
DI CAGLIARI

Introduction

Search problems

- ▶ Many real-world problems can be formulated as **searching** for a sequence of actions to attain a given goal
- ▶ **Rational agent** approach: **rigorous problem formulation** (goal, available actions, etc.)
- ▶ This course introduces to basic search algorithms that underlie the ones used in real-world applications

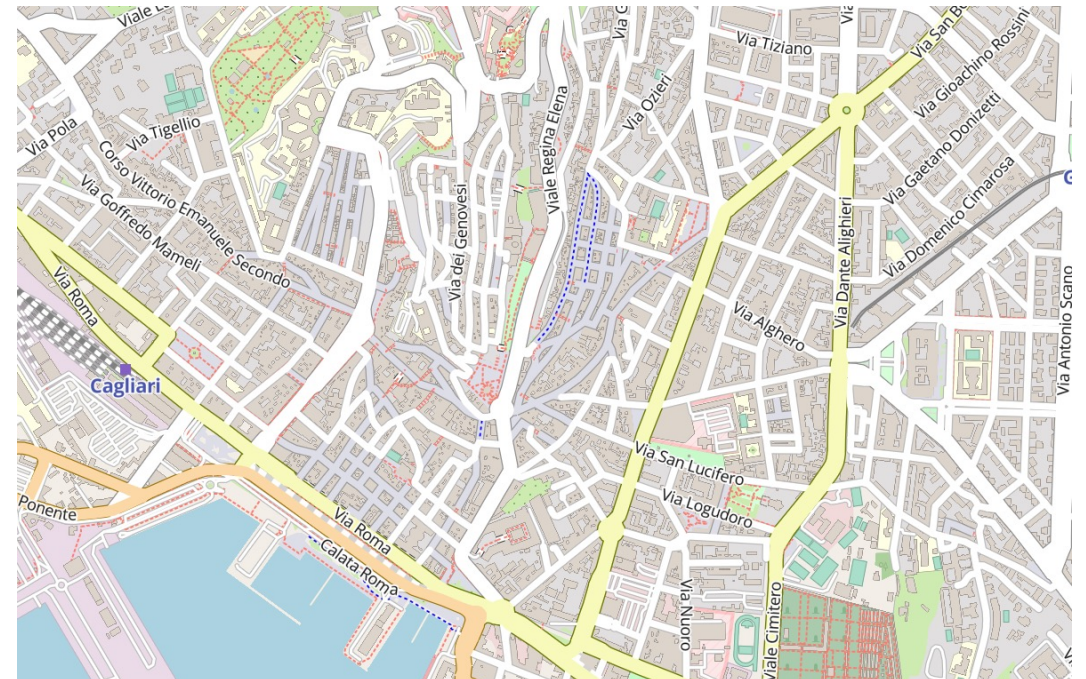
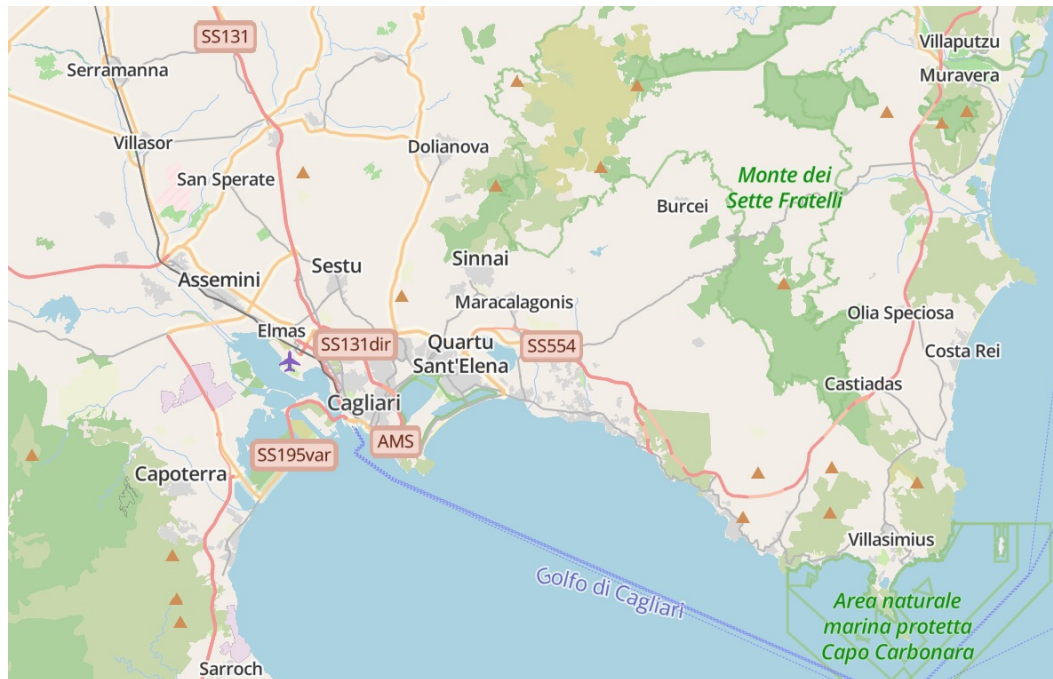


Application examples

- ▶ Route-finding
 - finding routes on maps
 - airline travel planning
 - routing in computer networks
 - ...
- ▶ Robot navigation
- ▶ VLSI design
 - cell layout
 - channel routing
- ▶ Games (draughts, chess, etc.)
- ▶ ...

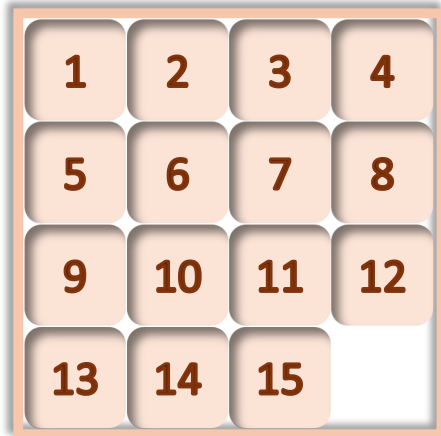
Example: route-finding

Finding a route (e.g., the shortest one) from a starting position to a destination one on a given map (usually in terms of a sequence of intermediate positions)

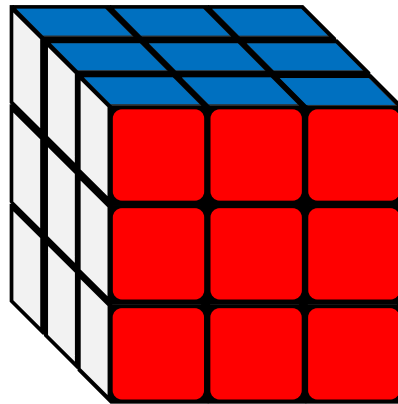


Example: games

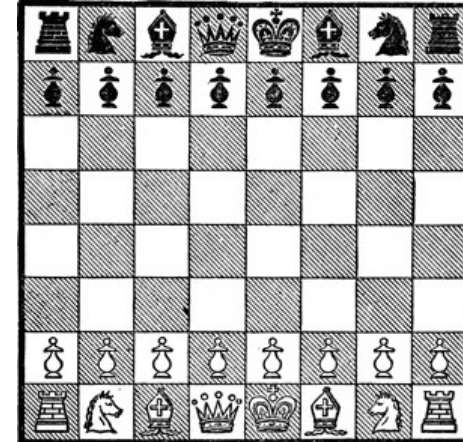
Finding a sequence of moves (e.g., the shortest one) that leads from an initial configuration to a desired one (for some problems there is a **set** of desired configurations, e.g., checkmate in chess)



15-puzzle: a classical AI's toy-problem



Rubik's cube



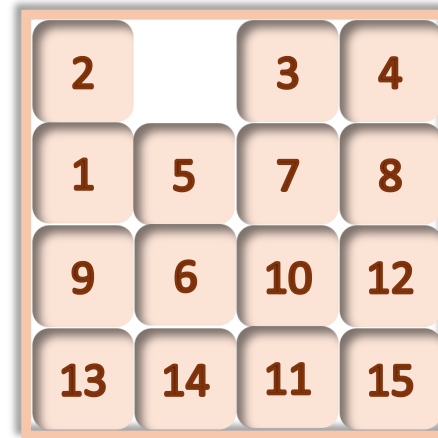
Chess

Common elements: "world states"

A set of possible problem-specific "states"



route-finding: a (usually finite) set of locations of interest in a given map



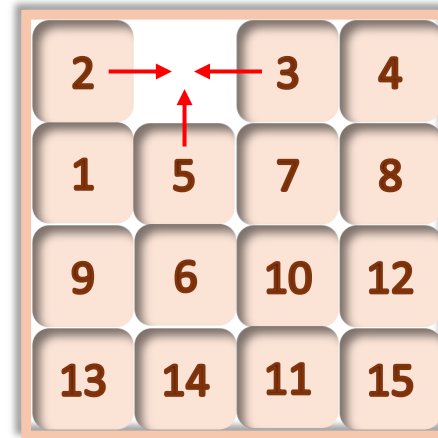
15-puzzle: any possible arrangement of the tiles

Common elements: actions

A set of **actions** that lead from one state to another



route-finding: moving from any location to **nearby** ones



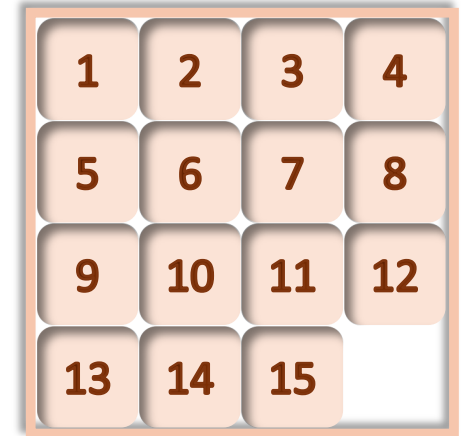
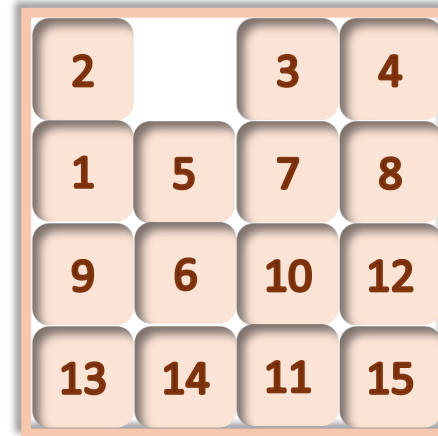
15-puzzle: sliding a tile to an **empty, adjacent** position

Common elements: goal

Goal: reaching a desired state D from a starting one S



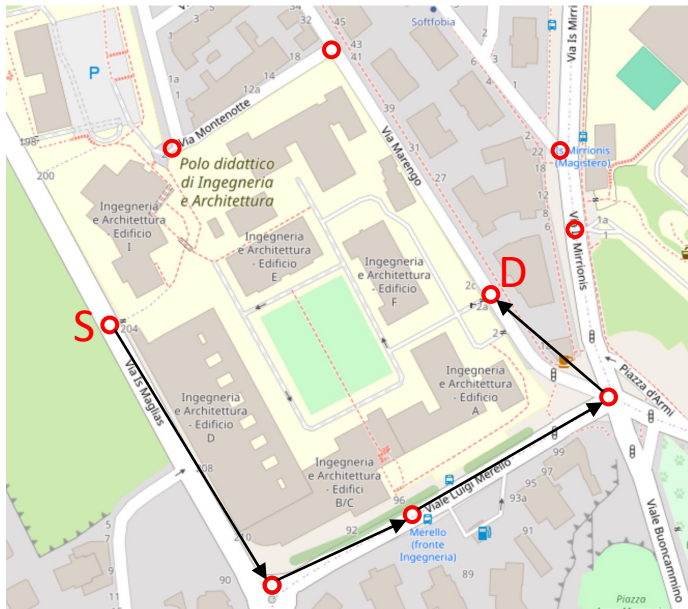
route-finding: reaching a destination D from a starting location S



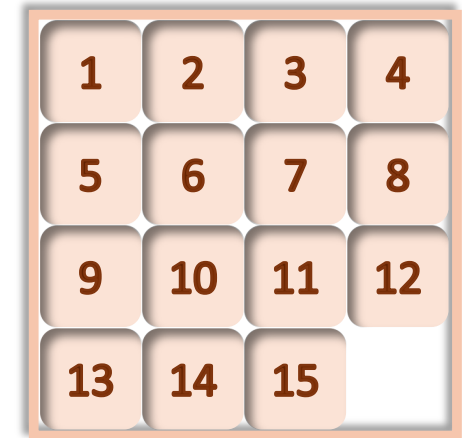
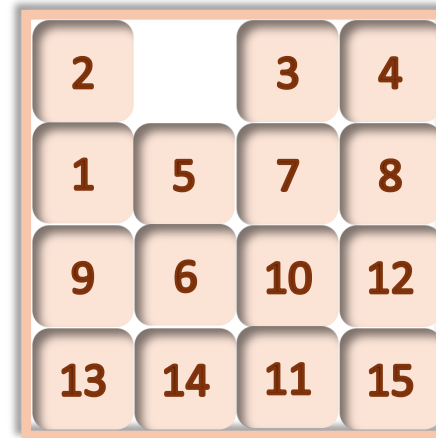
15-puzzle: reaching a desired configuration (e.g., the one on the right) from an initial one (left)

Common elements: solution

Solution: any sequence of actions leading from S to D



route-finding: a sequence of steps from S to D through nearby locations



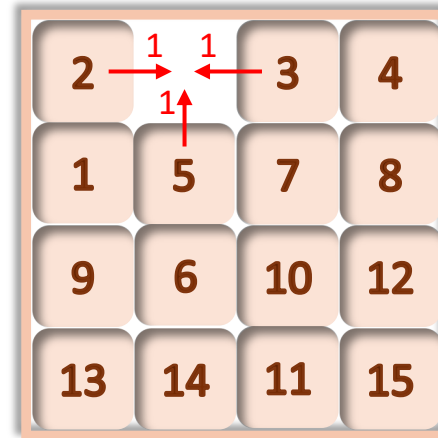
15-puzzle: a sequence of moves leading from the initial (left) to the desired configuration (right):
2→, 1↑, 5←, 6↑, 10←, 11↑, 15←

Common elements: cost of the actions

Usually, action execution has a **cost** (in a problem-specific unit of measurement)



route-finding: length of each movement, or duration, or fuel consumption, or...



15-puzzle: sliding any tile to an empty, adjacent position can be considered to have a cost equal to one move

Common elements: goal revisited

- ▶ Since executing actions has a cost, the goal is usually stated as finding the **minimum-cost solution** from S to D
- ▶ Examples
 - **route-finding**: finding the **shortest** (or **fastest**, or **cheaper**, or...) route from the starting location to the destination
 - **15-puzzle**: finding the **shortest** sequence of moves leading from the initial configuration to the desired one

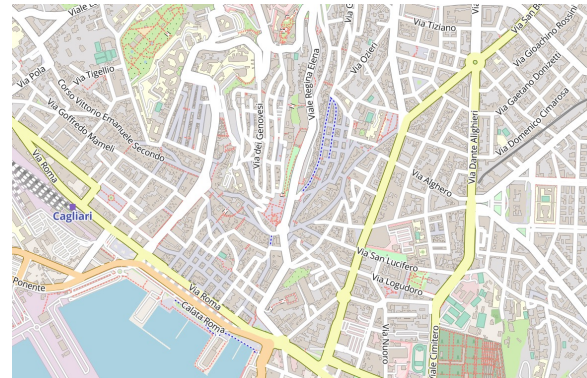
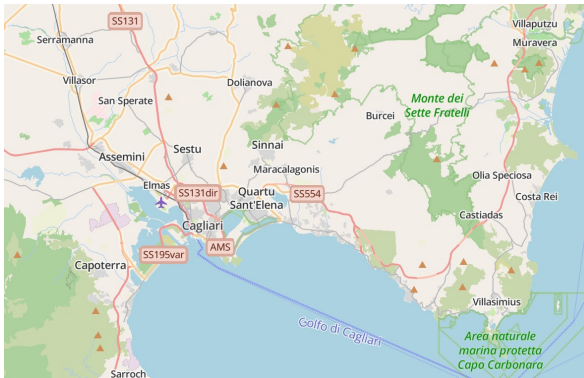
Problem formulation

- ▶ Apparently different problems share several common elements...
- ▶ This suggest a common, **abstract** problem **formulation**
- ▶ Crucial step: finding the proper **abstraction level**...
 - not straightforward for real-world problems
 - e.g., in route-finding:
 - considering actions like "moving two inches forward"?
 - road condition? weather? etc.
- ▶ Main elements of problem formulation
 - **states** and **operators**: the **state space**
 - **steps**, **paths** and **path cost**

States

The set of all possible "world states" of interest

- **15-puzzle**: each possible configuration of the tiles
- **route-finding**: being in one of a **finite** set of problem-specific locations of interest
 - cities, railway stations, road junctions, etc.
 - street numbers, crossings, squares, public places, bus stops, underground stations, etc.

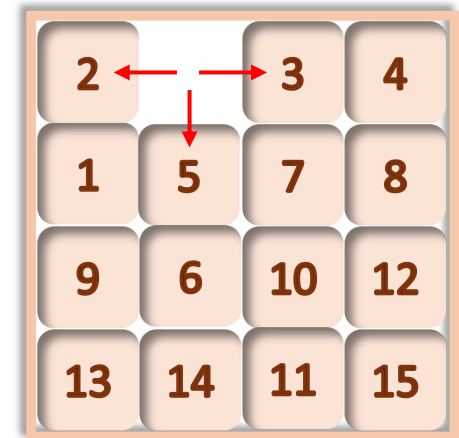
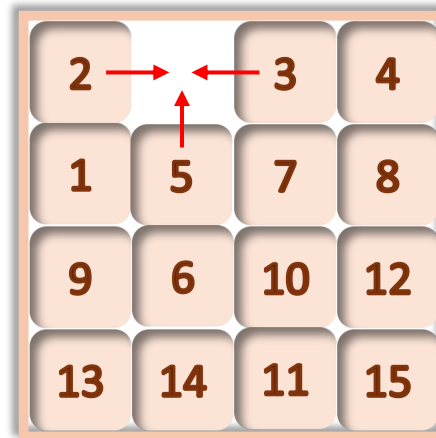


Operators

The set of available actions leading from one state to another



route-finding: moving between adjacent locations (states)



15-puzzle: two possible descriptions of the operators

- left: sliding any **tile** to an empty, adjacent position ($15 \times 4 = 60$ operators)
- right: sliding the **blank** to an adjacent position (4 operators, much more convenient)

State space

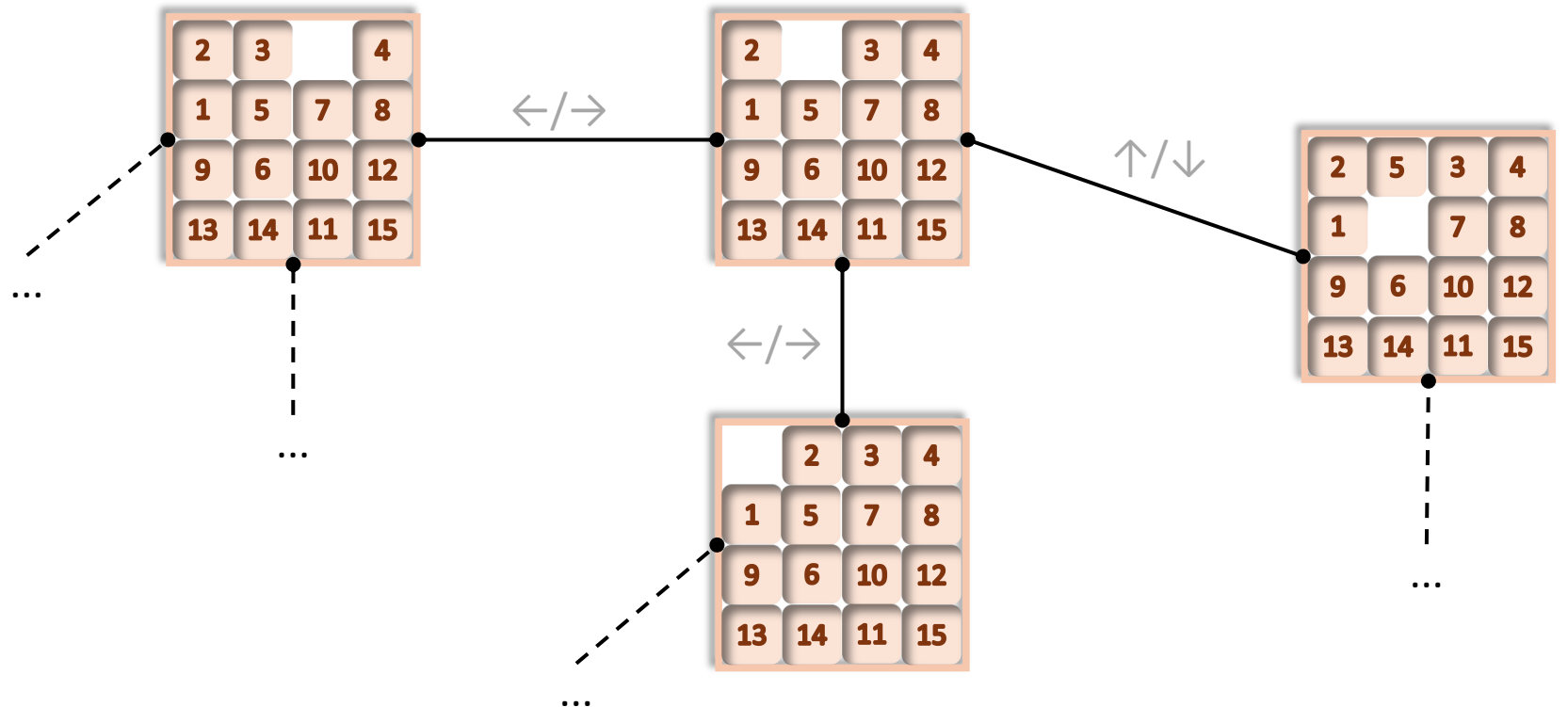
A **graph** where

- each **node** represents a distinct **state**
- each **edge** represents an **operator (action)** between the corresponding states
 - **undirected** edge: **reversible** action (e.g., 15-puzzle, two-way street by car)
 - **directed** edge: **non-reversible** action (e.g., one-way street by car)

State space: an example

A small portion of the **15-puzzle** state space

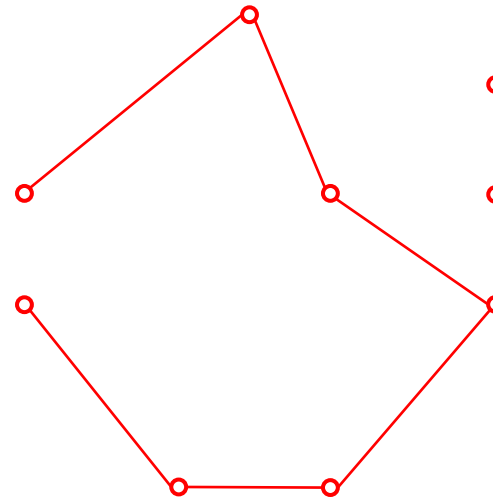
- reversible actions: undirected edges



State space: an example

Route-finding: the map itself can be seen as a graph...

The state space is a **topological** (not to scale) representation of a map

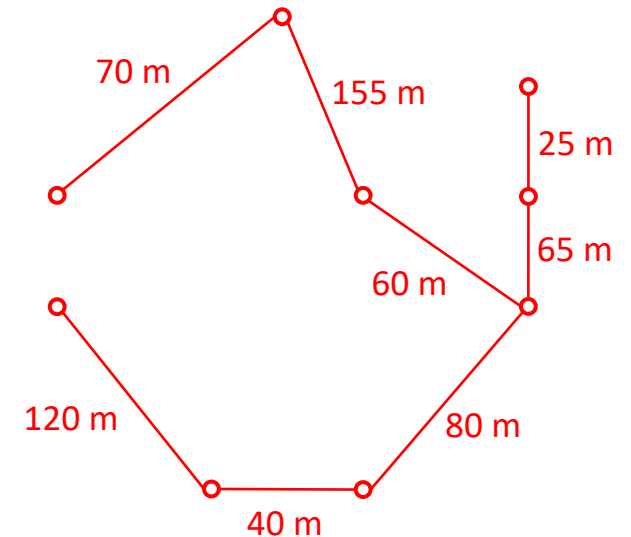


Step cost

The cost of each operator (in a problem-specific unit of measurement)

- "**step**" refers to a movement between two **adjacent** states in the state space (i.e., the execution of the corresponding action)

example on **route-finding**, assuming
the cost of interest to be the **length**
(in meters) of a given movement

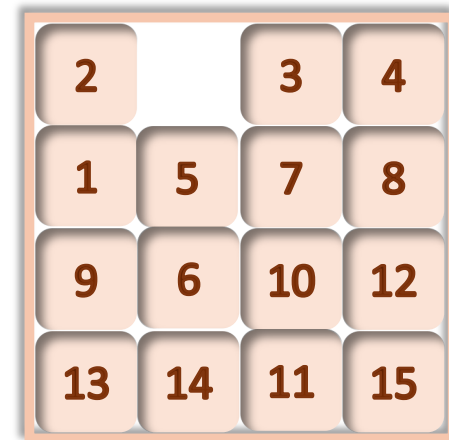
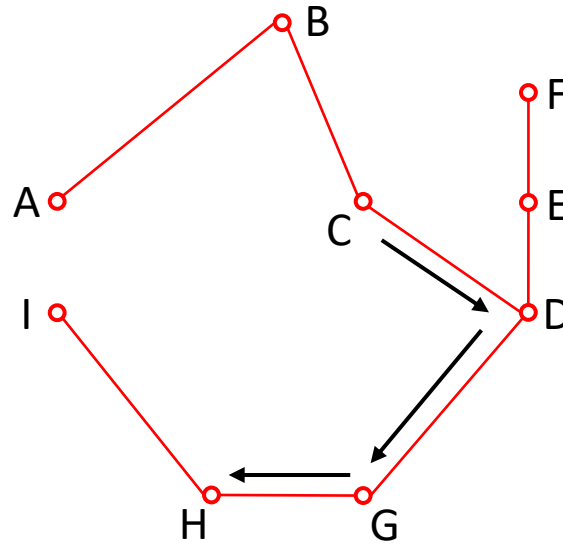


Path

A sequence of states reachable by a sequence of steps in the state space



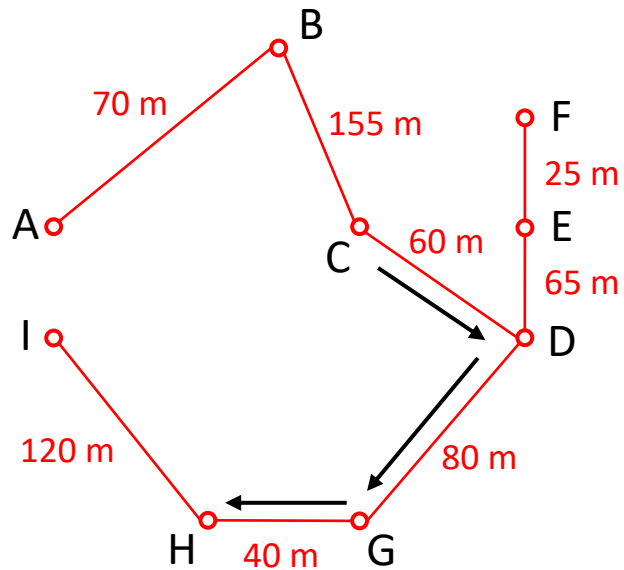
a possible path in the **route-finding** state space: C, D, G, H



steps of a possible path in the **15-puzzle** state space, starting from the above configuration: $\rightarrow \downarrow \leftarrow \downarrow$

Path cost

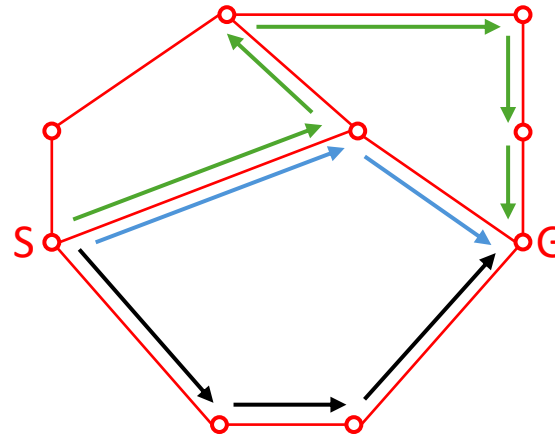
Cost of the actions in a path (usually, the sum of the corresponding step costs)



example on **route-finding**:
the cost of the path C, D, G, H
is $60 + 80 + 40 = 180$ m

Solutions and goal

- ▶ Solution: **any** path from a starting state S to a desired state G
 - there can be different paths between any pair of nodes of the state space...
 - an example for a generic state space:



- ▶ Goal: finding the **minimum-cost** solution

Properties of search problems

- ▶ We have seen a common problem formulation
- ▶ However there exist several kinds of search problems, with different degrees of complexity...
- ▶ Before presenting algorithms to solve them, let us look at their main **properties**

Properties of search problems

▶ **Static vs dynamic**

- does the environment **change** during action **execution**?
- e.g.: 15-puzzle is static, route-finding is dynamic (road closures, etc.)

▶ **Fully vs partially observable**

- is the **current** state completely known?
- e.g.: 15-puzzle is fully observable, robot navigation is partially observable (occlusions, noisy sensor data, etc.)

▶ **Discrete vs continuous** (states and operators)

- e.g.: 15-puzzle is discrete, robot navigation can be continuous

▶ **Deterministic vs non-deterministic**

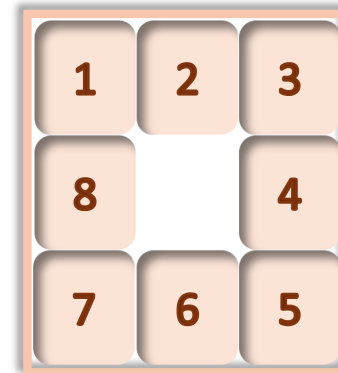
- is the **outcome** (end state) of any action sequence **certain**?
- e.g.: 15-puzzle is deterministic, chess is not (**two-player** game...)

Focus of this course

- ▶ The **simplest** kind of search problem
 - static
 - fully observable
 - discrete
 - deterministic
- ▶ They can be solved in two **distinct** steps
 - **searching** for a solution ("offline", **before** action execution)
 - **executing** the corresponding actions
- ▶ We shall also focus on the **first** step: **searching** for a solution
- ▶ Two application examples: 8-puzzle, route-finding

Application example: 8-puzzle

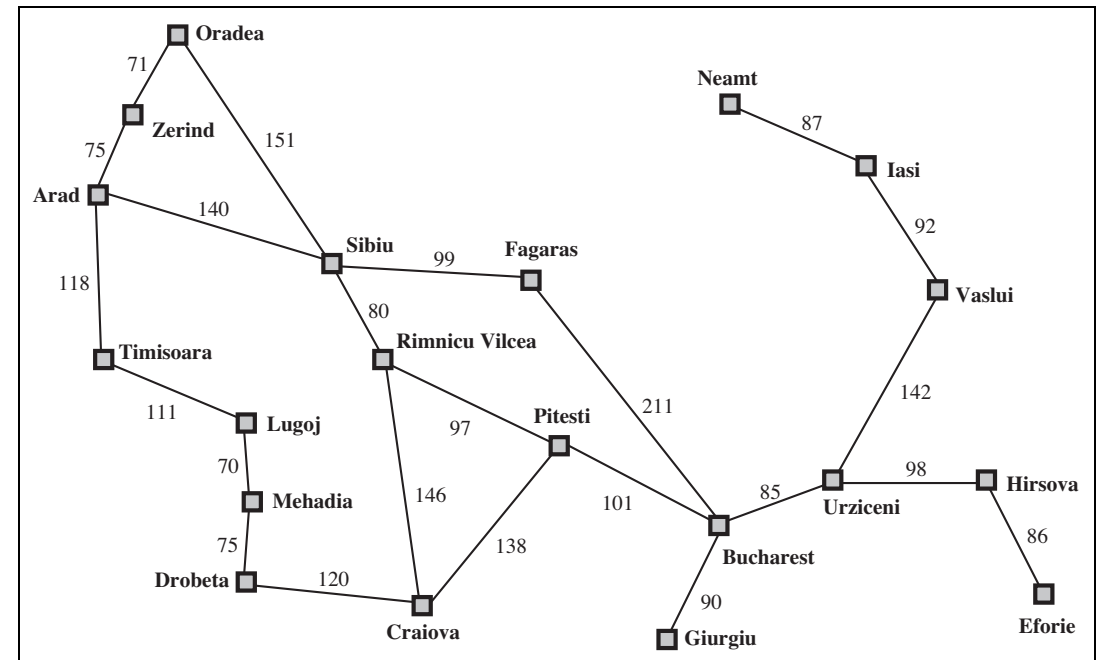
- ▶ A simpler version of 15-puzzle (but challenging enough...)
- ▶ Some facts
 - size of the state space: $9! = 362,880...$
 - only **half** of the states are reachable from any given state
 - only half of all possible problem instances are therefore **solvable** (checking for solvability is easy)



Application example: route-finding

State space of a simple route-finding problem

- twenty Romanian cities
- several two-way roads connect some pairs of cities (the state space is a **connected** graph: each city can be reached from each other)
- goal: finding the **shortest** route between any given pair of cities
- step cost: road length in Km (shown on each edge)
- static, deterministic formulation (no road closures, etc.): all possible paths are feasible



Searching for solutions

Basic idea

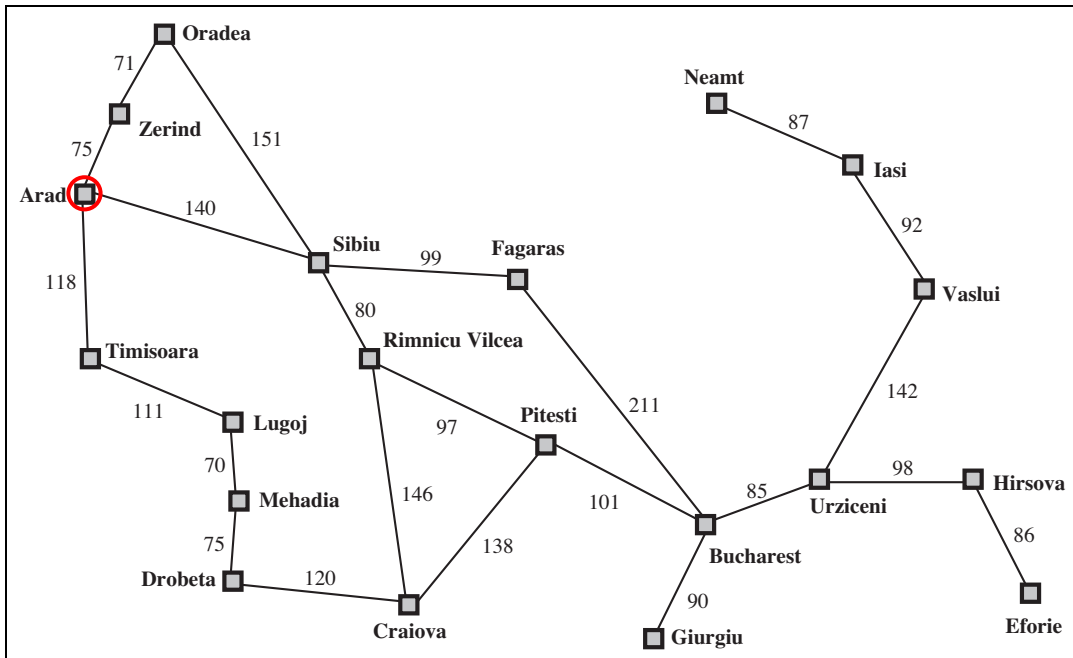
- ▶ Generating **all** possible paths from the initial state S , until one ending in the goal state G is found (hopefully, the minimum-cost path)
- ▶ Informal procedure
 - starting point: the initial state S (zero-length path)
 - repeat
 - **choose** one of the current paths p
 - extend p by adding to it **each** possible step
 - until a path ending in G is obtained

An example on route-finding

Goal: getting from Arad to Bucharest

Initial path (zero length):

— Arad

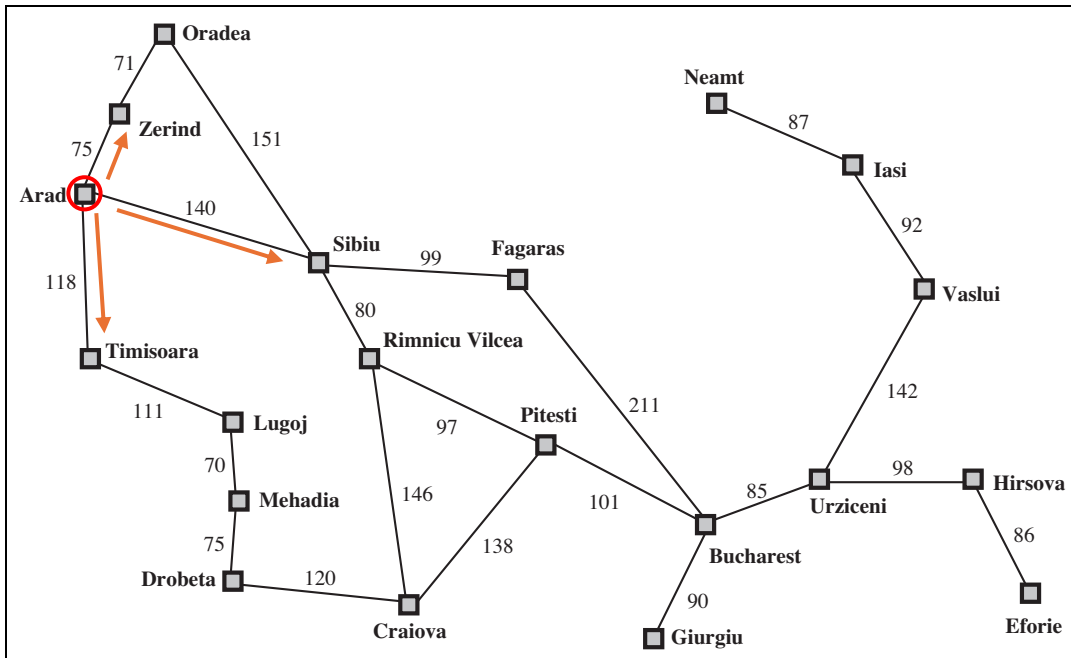


An example on route-finding

Goal: getting from Arad to Bucharest

Paths after the 1st iteration:

- Arad, Sibiu
- Arad, Zerind
- Arad, Timisoara

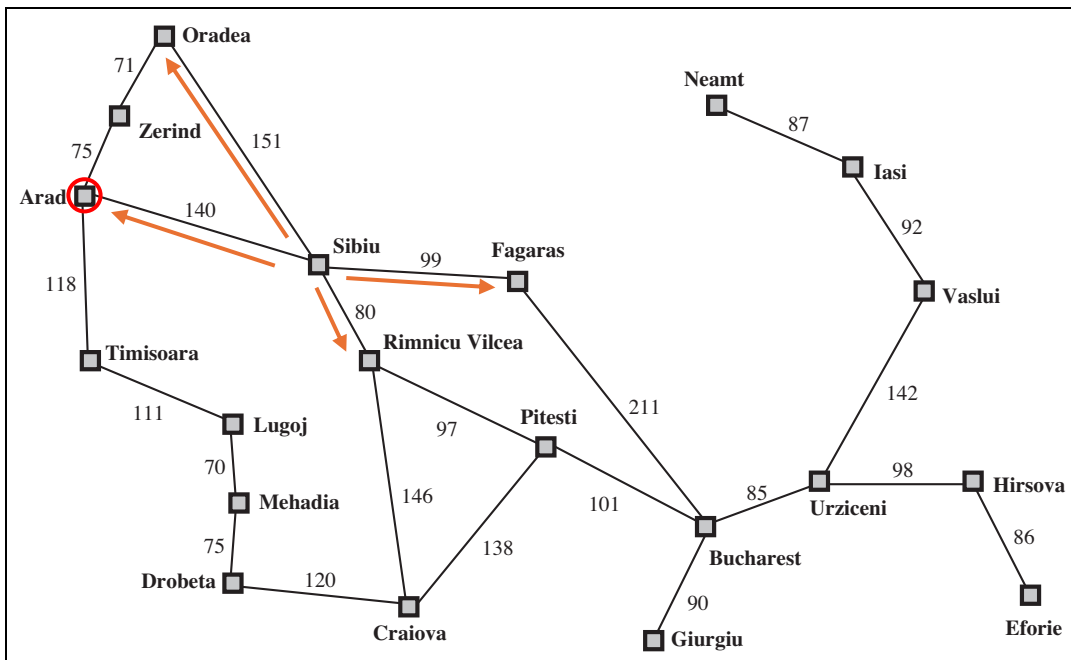


An example on route-finding

Goal: getting from Arad to Bucharest

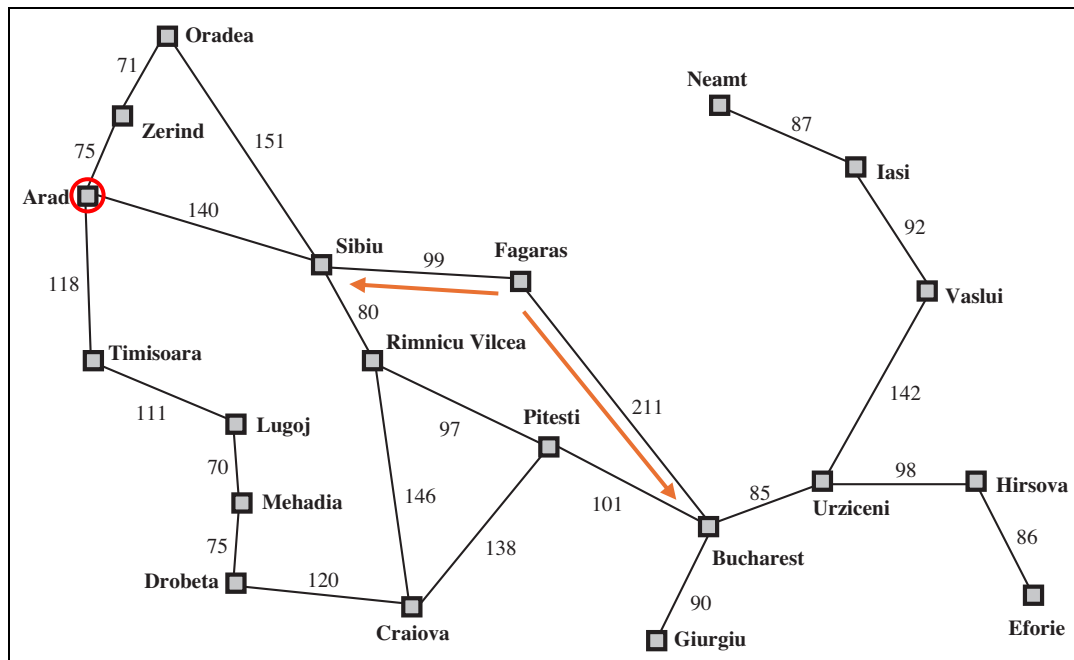
Paths after the 2nd iteration, assuming (Arad, Sibiu) is extended:

- Arad, Zerind
- Arad, Timisoara
- Arad, Sibiu, Fagaras
- Arad, Sibiu, Oradea
- Arad, Sibiu, Rimnicu Vilcea
- Arad, Sibiu, Arad



An example on route-finding

Goal: getting from Arad to Bucharest



Paths after the 3rd iteration, assuming (Arad, Sibiu, Fagaras) is extended:

- Arad, Zerind
- Arad, Timisoara
- Arad, Sibiu, Oradea
- Arad, Sibiu, Rimnicu Vilcea
- Arad, Sibiu, Arad
- Arad, Sibiu, Fagaras, Bucharest
- Arad, Sibiu, Fagaras, Sibiu

One path ending in the goal state has been found

Towards a search algorithm

- ▶ Does the above approach look "intelligent"?
 - if not, can you find a better one...?
 - anyway, the question is ill-posed: we are considering **rational** agents
- ▶ A more meaningful question: how **effective** and **efficient** is it?
- ▶ To answer this question one needs to better describe this approach as a formal **algorithm** – to this aim, one has to devise
 - how to **represent** the set of paths that are iteratively generated
 - how to **systematically generate** all possible paths from the initial state

Representing paths: the *search tree*

Paths that fan out from the initial state can be represented as a **tree graph**, called **search tree**

- each **node** is associated to one state of a **distinct** path from the initial state
- **root** node: initial state
- **inner** node: a non-end state of a path
- **leaf** node: end state of a path
- **edge**: operator leading from the parent's to the child node's state
- **depth** of a node: length of a path, i.e., number of operators (edges) in the path from the root (by convention the **root** has **zero** depth)

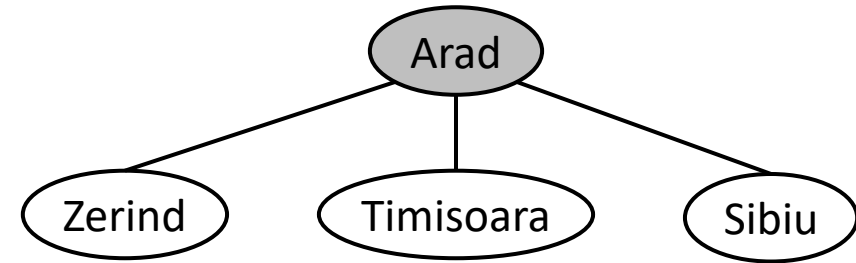
Search tree: an example

How the search tree is constructed in the previous example

initialisation



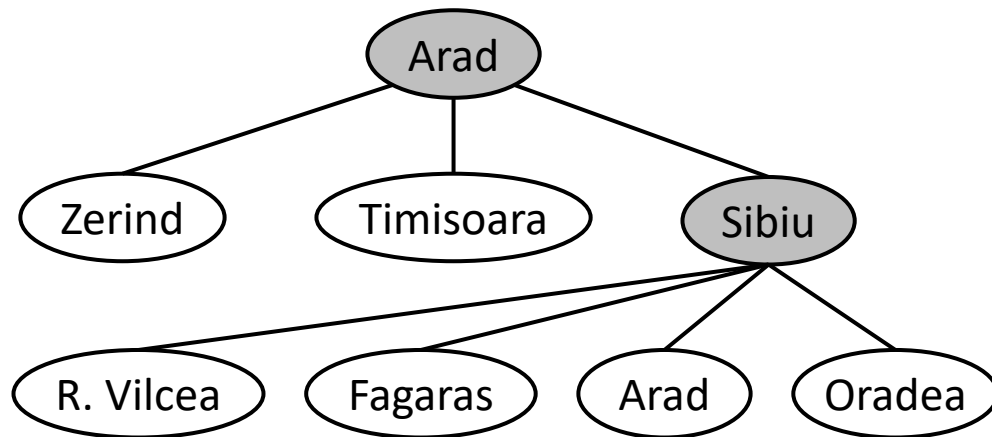
first iteration



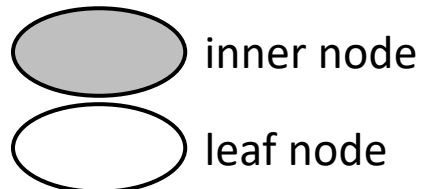
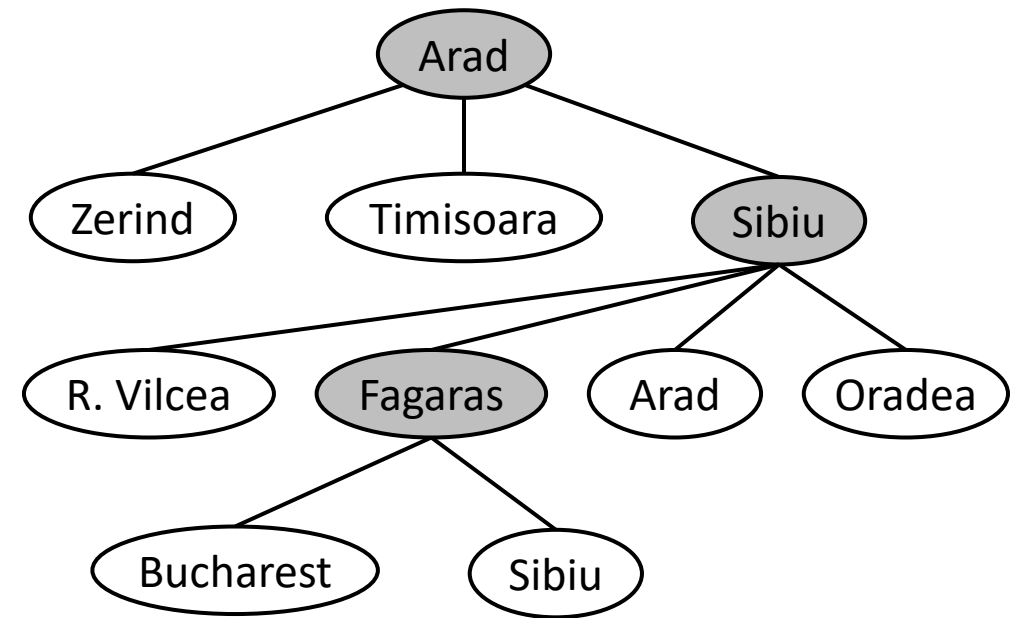
Search tree: an example

How the search tree is constructed in the previous example

second iteration



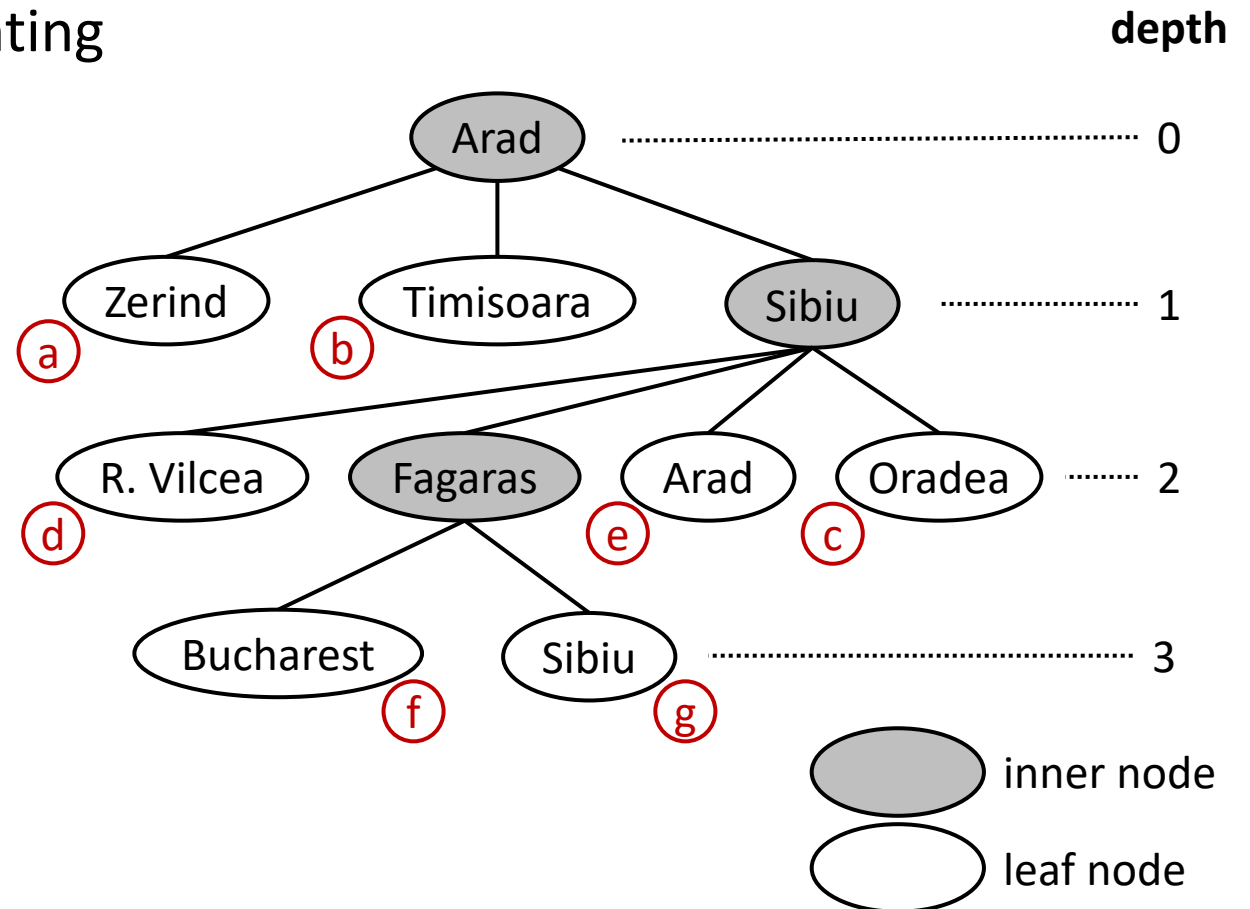
third iteration



Search tree: an example

There are seven **leaf** nodes representing seven **distinct** paths

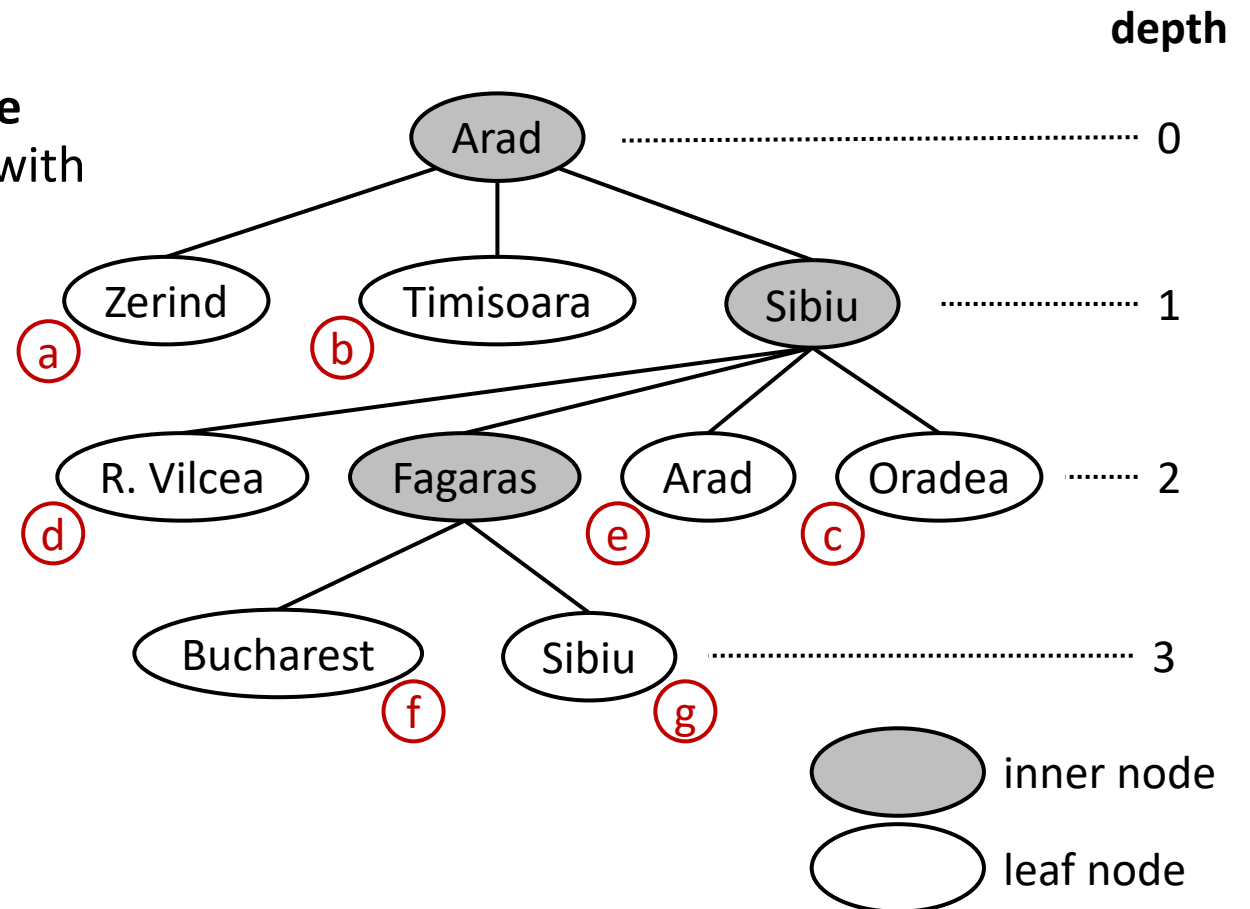
- a) Arad, Zerind
- b) Arad, Timisoara
- c) Arad, Sibiu, Oradea
- d) Arad, Sibiu, R. Vilcea
- e) Arad, Sibiu, Arad
- f) Arad, Sibiu, Fagaras, Bucharest
- g) Arad, Sibiu, Fagaras, Sibiu



Search tree: an example

Remarks

- **distinct** nodes associated to the **same** state (two nodes with Arad and two with Sibiu) represent **different** paths
- **cyclical** paths or **loops** (**e** and **g**) are useless: they can be **discarded**



A basic tree-search algorithm

Pseudo-code of a basic tree-search algorithm that **systematically** generates **all** possible paths from the initial state, and represents them as a search tree – a process called search tree **expansion**

```
function Tree-Search(problem) returns a solution, or failure
    generate the root node using the initial state of problem
    loop do
        if there are no leaf nodes
            return failure
        choose a leaf node n
        if n contains a goal state
            return the corresponding solution
        expand n
```

A basic tree-search algorithm

argument: a **data structure** containing all the information on the search problem: initial state, description of the goal state(s), operators, etc.

initialisation: the root node is associated to the initial state; it is currently also a **leaf** node

main loop: each iteration **expands one** of the current leaf nodes (if any), until a solution is found or there are no more leaf nodes to expand (in which case the algorithm terminates with a failure)

```
function Tree-Search(problem) returns a solution, or failure
    generate the root node using the initial state of problem
    loop do
        if there are no leaf nodes
            return failure
        choose a leaf node n
        if n contains a goal state
            return the corresponding solution
        expand n
```

node expansion: all the successors of the state associated to the chosen node (reachable by a **single** operator) are generated and are added as leaf nodes to the search tree (cyclical paths can be discarded)

An example on 8-puzzle

- ▶ A possible (solvable) problem instance:

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

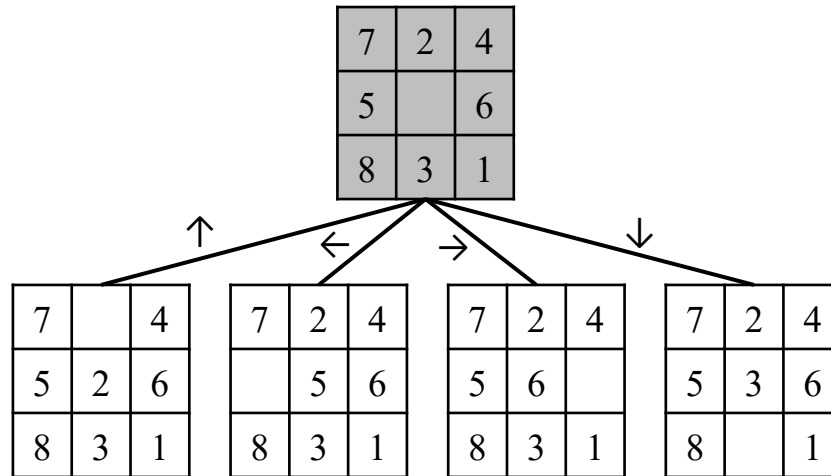
- ▶ A few iterations of the search tree construction process are shown next
 - cyclical paths are discarded
 - the leaf node to expand at each iteration is randomly chosen
 - the operators (moves of the blank) are indicated on the edges

An example on 8-puzzle

initialisation

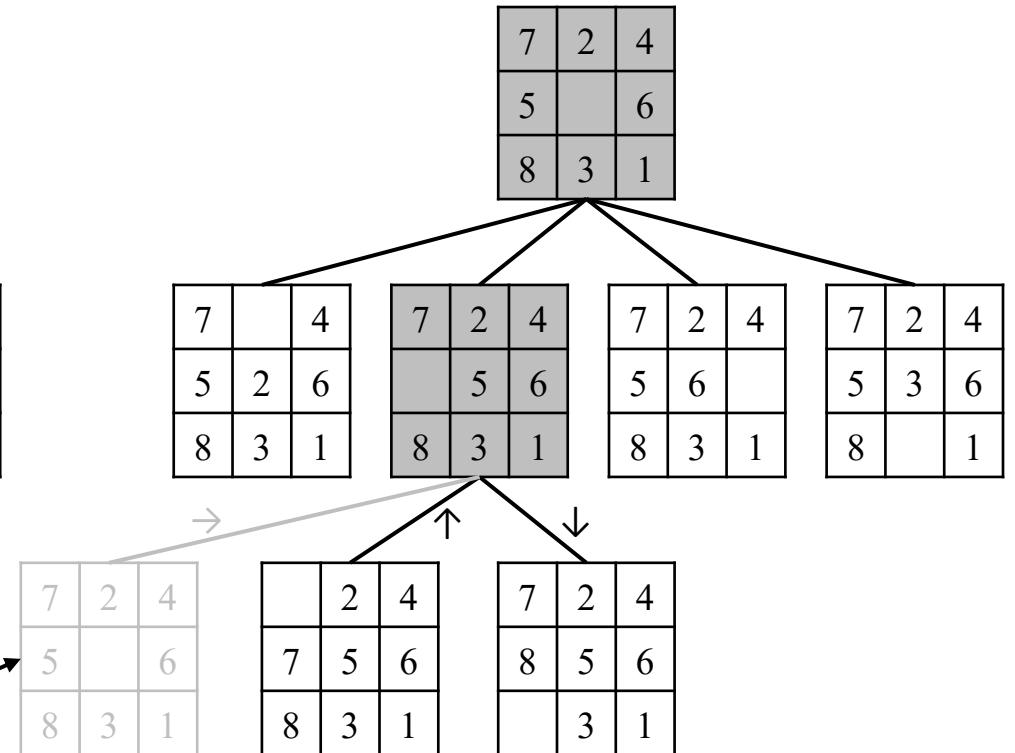
7	2	4
5		6
8	3	1

first iteration



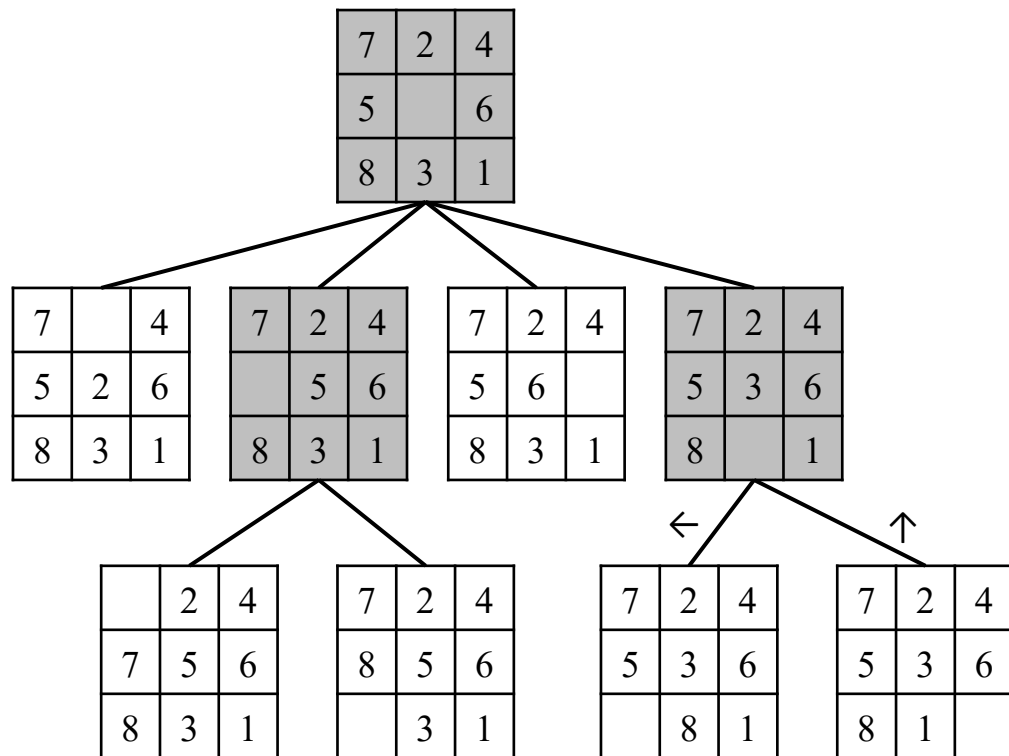
second iteration

this node is **discarded**: it
would lead to a cyclical path
(same state as the root node)

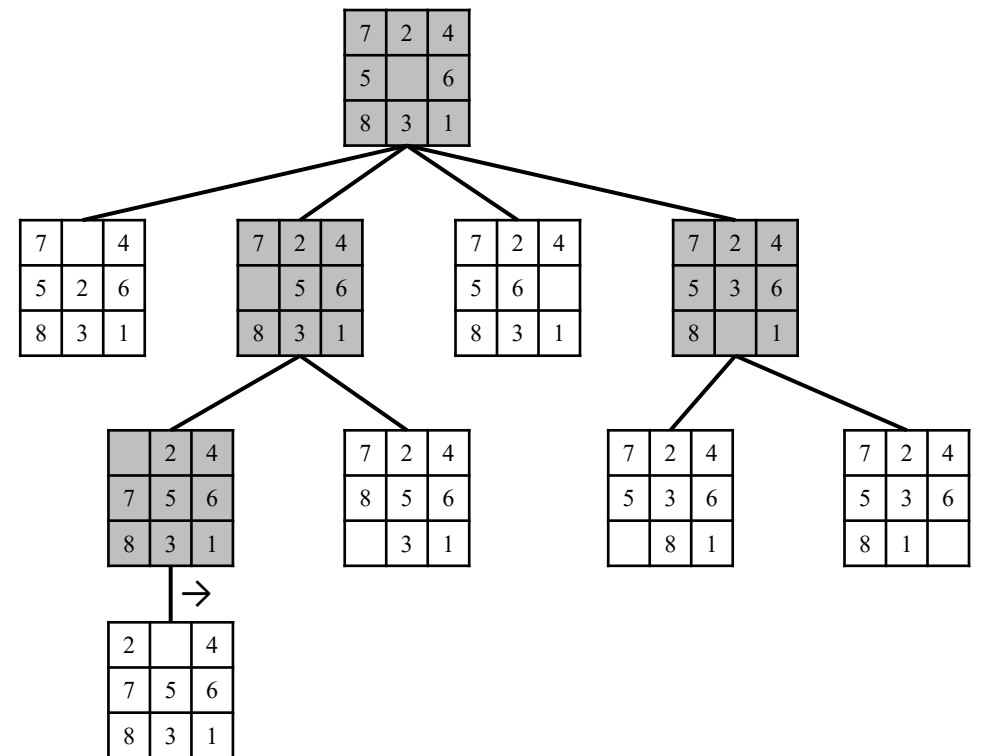


An example on 8-puzzle

third iteration

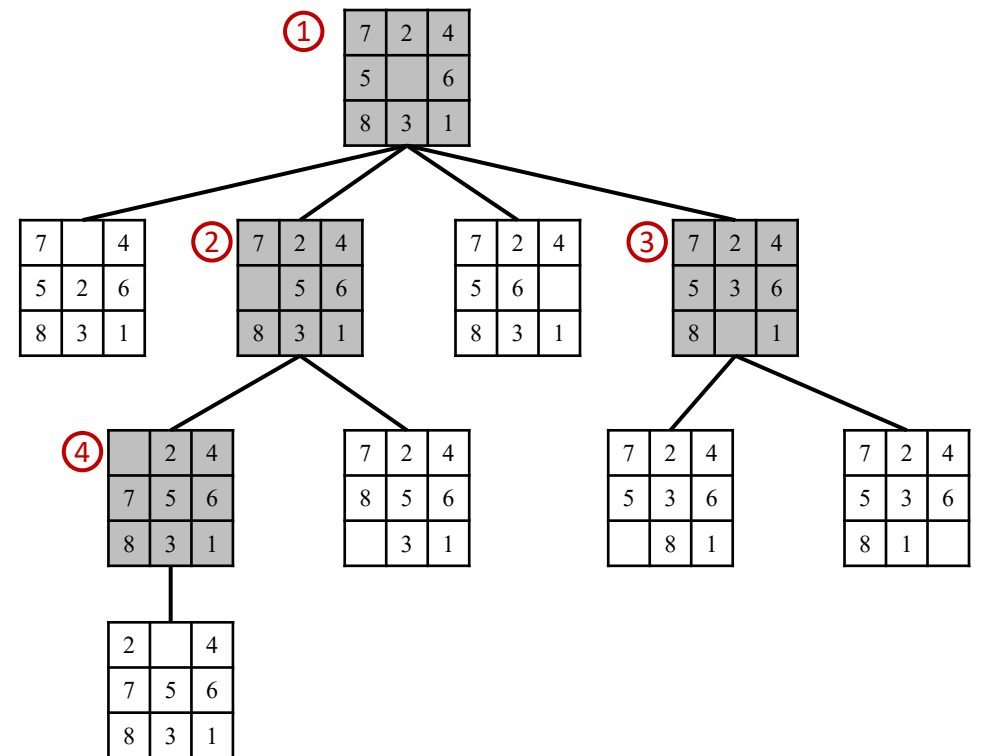


fourth iteration



Search tree construction process

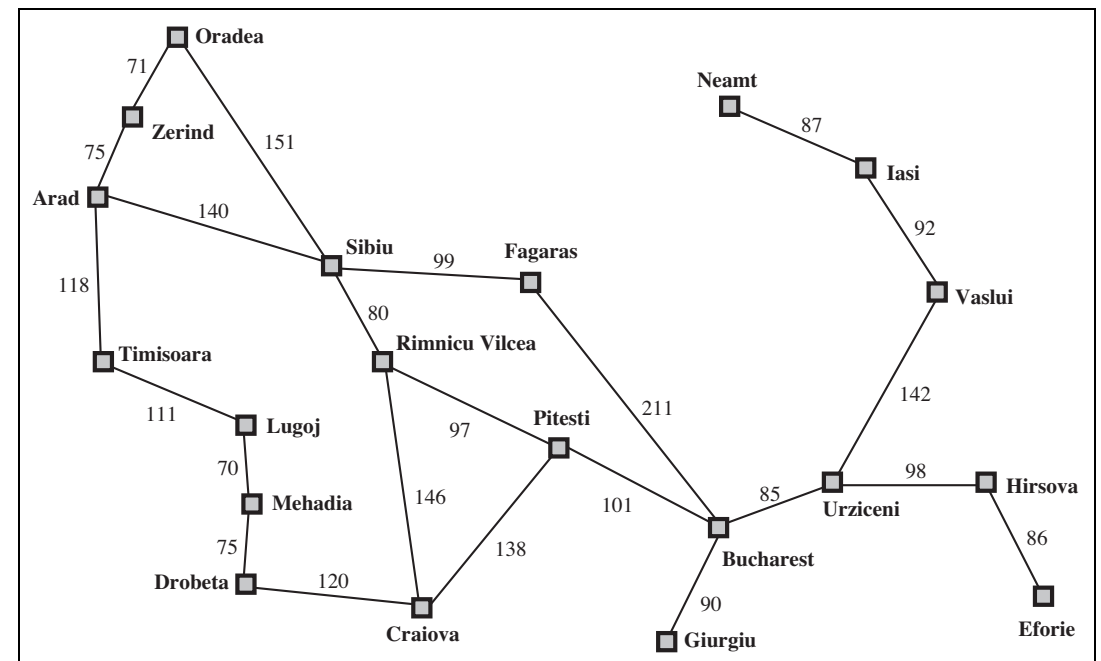
A concise representation of the search tree construction process, indicating which node was expanded at each iteration



Exercise

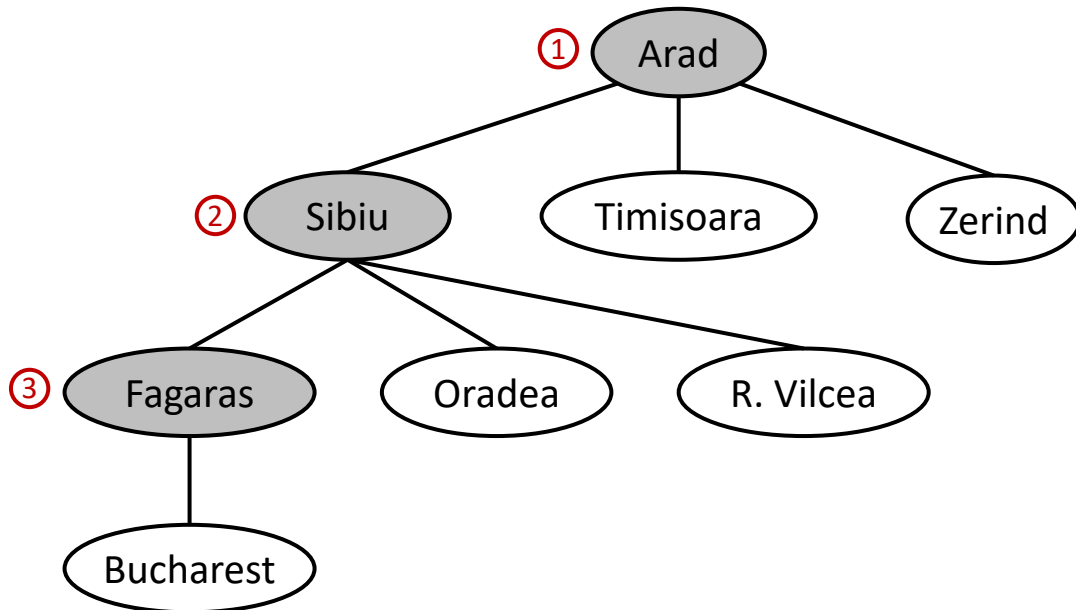
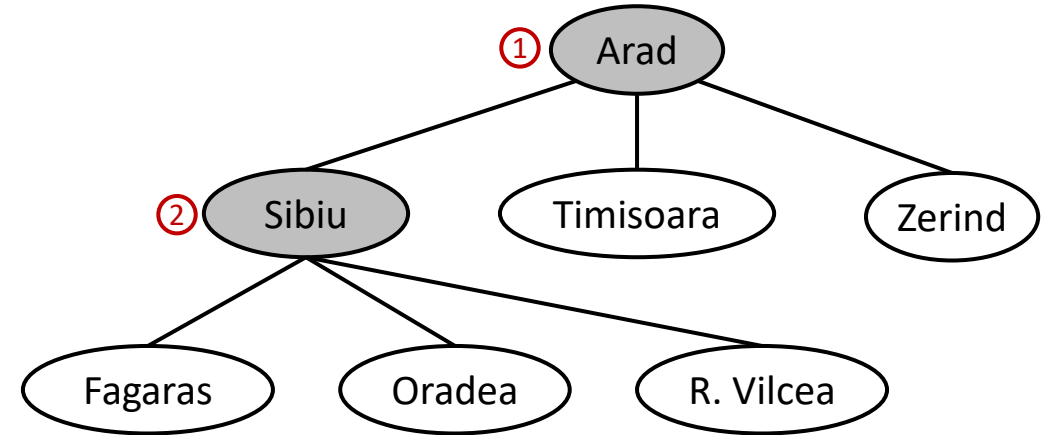
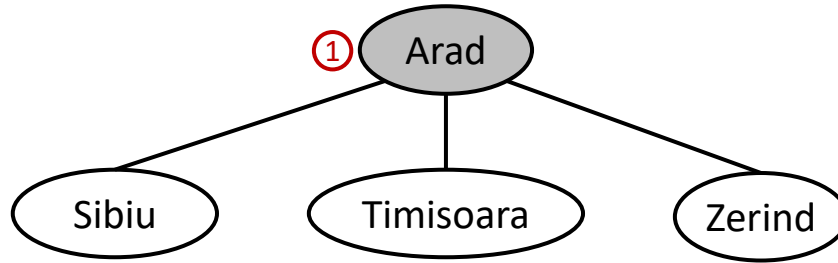
Draw the search tree obtained after **three** iterations to find a route from Arad to Vaslui in the Romania map

- avoid cyclical paths
- choose the node to expand next according to **alphabetical** order among city names (to allow your solution to be compared with the correct one)



Solution

initialisation



Implementation example (part I)

Introduction

- ▶ A **basic** Python implementation of tree-search algorithms is proposed
 - source code available as a [Python notebook](#)
 - advanced implementations can be found, e.g., in the [Code](#) section of the course textbook's web page (<https://aima.cs.berkeley.edu/>)
 - example problems: 8-puzzle and route-finding on the Romania map
- ▶ Let us first focus on **data structures** for representing
 - states
 - operators
 - search trees

State representation

► Route-finding

- states correspond to possible locations (cities) in the map
- they can be simply represented by **strings** containing the city names (assuming they are all distinct), e.g.: "Arad", "Sibiu", "Bucharest"

► 8-puzzle

- any configuration of the eight tiles can be represented by a 3×3 **array** (e.g., nested **tuples**) containing the integers 0, 1, ..., 8 (0 denotes the blank)
- a single **tuple** can also be used

7	2	4
5		6
8	3	1

((7, 2, 4), (5, 0, 6), (8, 3, 1))
(7, 2, 4, 5, 0, 6, 8, 3, 1)

Successor function

- ▶ The successors of any given state can be returned by a **successor function** as a list:

`successors(<state>) → [<successor>, ... ]`

- ▶ Each successor can be represented by a dictionary:

`{"operator": <op>, "state": <state>, "step cost": <cost>}`

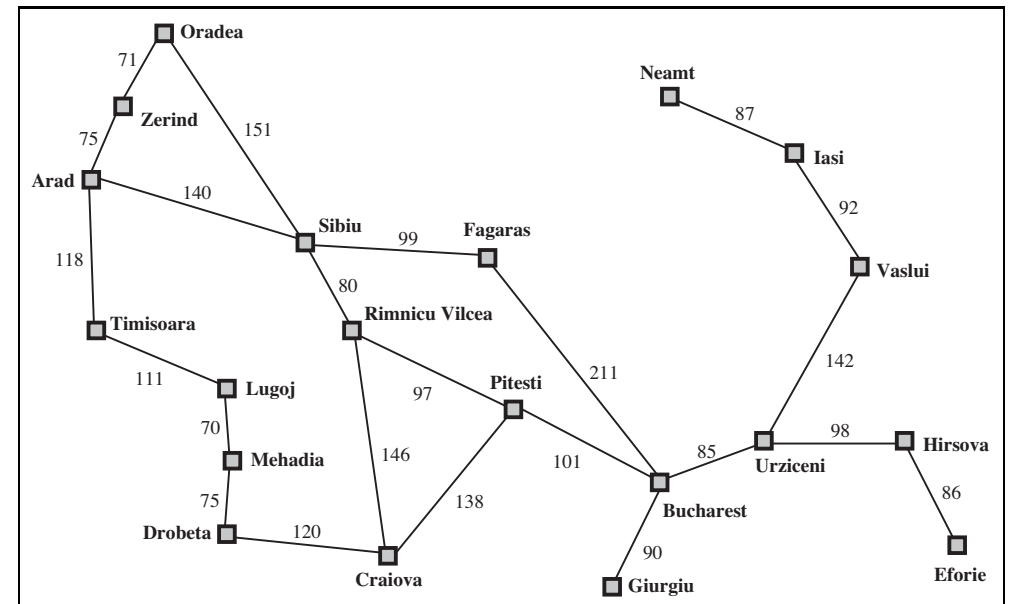
Implementing operators: route-finding

- ▶ Operators: moving from any given city to **adjacent** ones
- ▶ For each state (city) the adjacent cities and the corresponding costs should be accessed by the successor function, possibly through a suitable data structure
- ▶ This amounts to **explicitly** store the state space

Implementing operators: route-finding

A possible representation of the state space as nested **dictionaries**

```
{"Arad": {"Timisoara": 118, "Sibiu": 140, "Zerind": 75},  
  "Zerind": {"Arad": 75, "Oradea": 71},  
  "Oradea": {"Zerind": 71, "Sibiu": 151},  
  ...  
  "Eforie": {"Hirsova": 86}}
```



Implementing operators: route-finding

Example: the successors of the state "Arad":

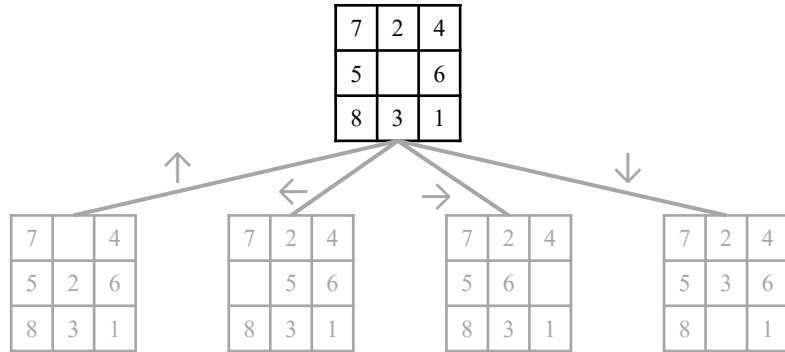
successors("Arad") →

```
[{"operator": "Arad-to-Sibiu", "state": "Sibiu", "step cost": 140},  
 {"operator": "Arad-to-Timisoara", "state": "Timisoara", "step cost": 118},  
 {"operator": "Arad-to-Zerind", "state": "Zerind", "step cost": 75}]
```

Implementing operators: 8-puzzle

- ▶ State space size: $9!$...
- ▶ **Explicit** representation: **not feasible**
- ▶ Fortunately not even necessary: the successors of any state can be easily **computed** by the successor function
 - the implementation is left as an **exercise** (**not** an `if-elif-else` cascade of explicit conditions for all possible blank positions, please)
 - some examples are shown next

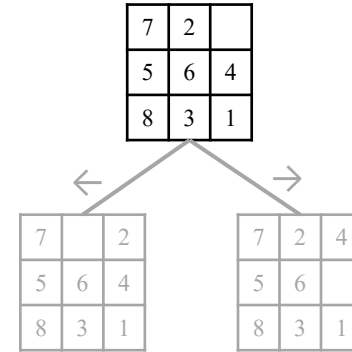
Example



successors((7, 2, 4, 5, 0, 6, 8, 3, 1))

→

```
[{"operator": "up",  
  "state": (7, 0, 4, 5, 2, 6, 8, 3, 1),  
  "step cost": 1},  
 ... ]
```



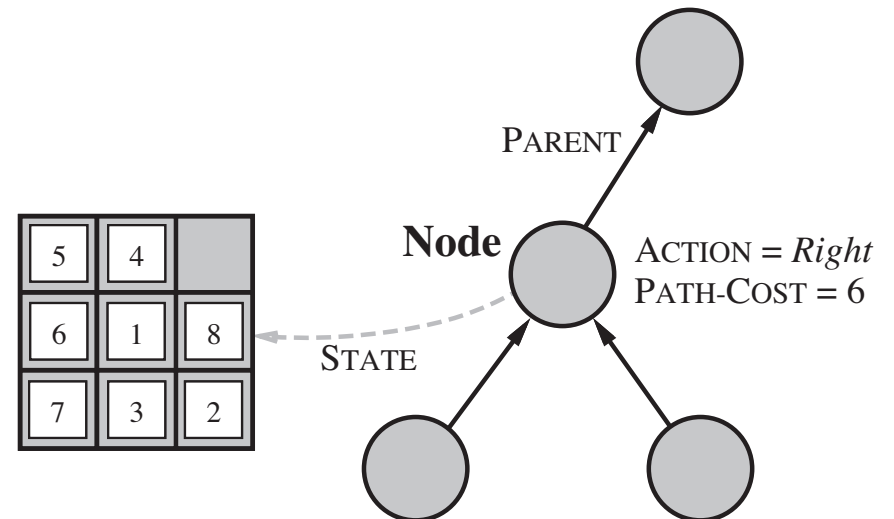
successors((7, 2, 0, 5, 6, 4, 8, 3, 1))

→

```
[{"operator": "left",  
  "state": (7, 0, 2, 5, 6, 4, 8, 3, 1),  
  "step cost": 1},  
 {"operator": "right",  
  "state": (7, 2, 4, 5, 6, 0, 8, 3, 1),  
  "step cost": 1}]
```

Representing a search tree

- ▶ The search tree structure (**nodes** and **edges**) is **not** problem-specific
- ▶ Since a node represents the **last** state of a **path** from the initial state, the basic information it should store is
 - the corresponding **state**
 - the **operator** from the parent node
 - the **path cost**
 - a reference (pointer) to the **parent node** (references to **child** nodes are not necessary: we will see why)



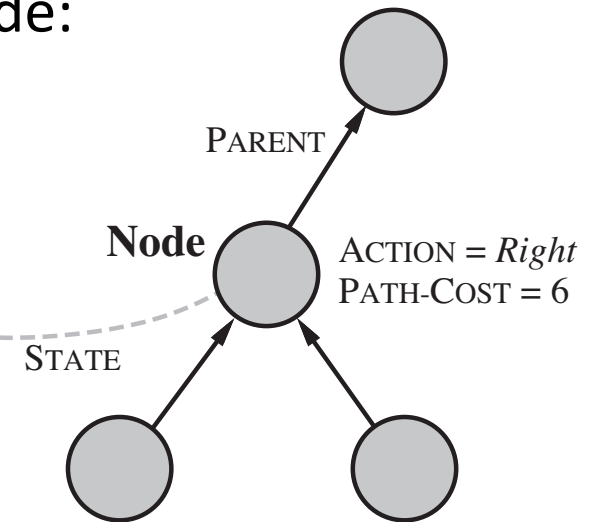
Example

Basic data structure for implementing a node: a **dictionary**

- assuming the variable `p` was associated to the parent node:

```
{"state":      (5, 4, 0, 6, 1, 8, 7, 3, 2),  
  "action":    "right",  
  "path_cost": 6,  
  "parent":    p}
```

5	4	
6	1	8
7	3	2



- for the **root** node the action and the reference to the parent node can be set to `None`:

```
{"state": ..., "action": None, "path cost": 0, "parent": None}
```

Referencing a search tree

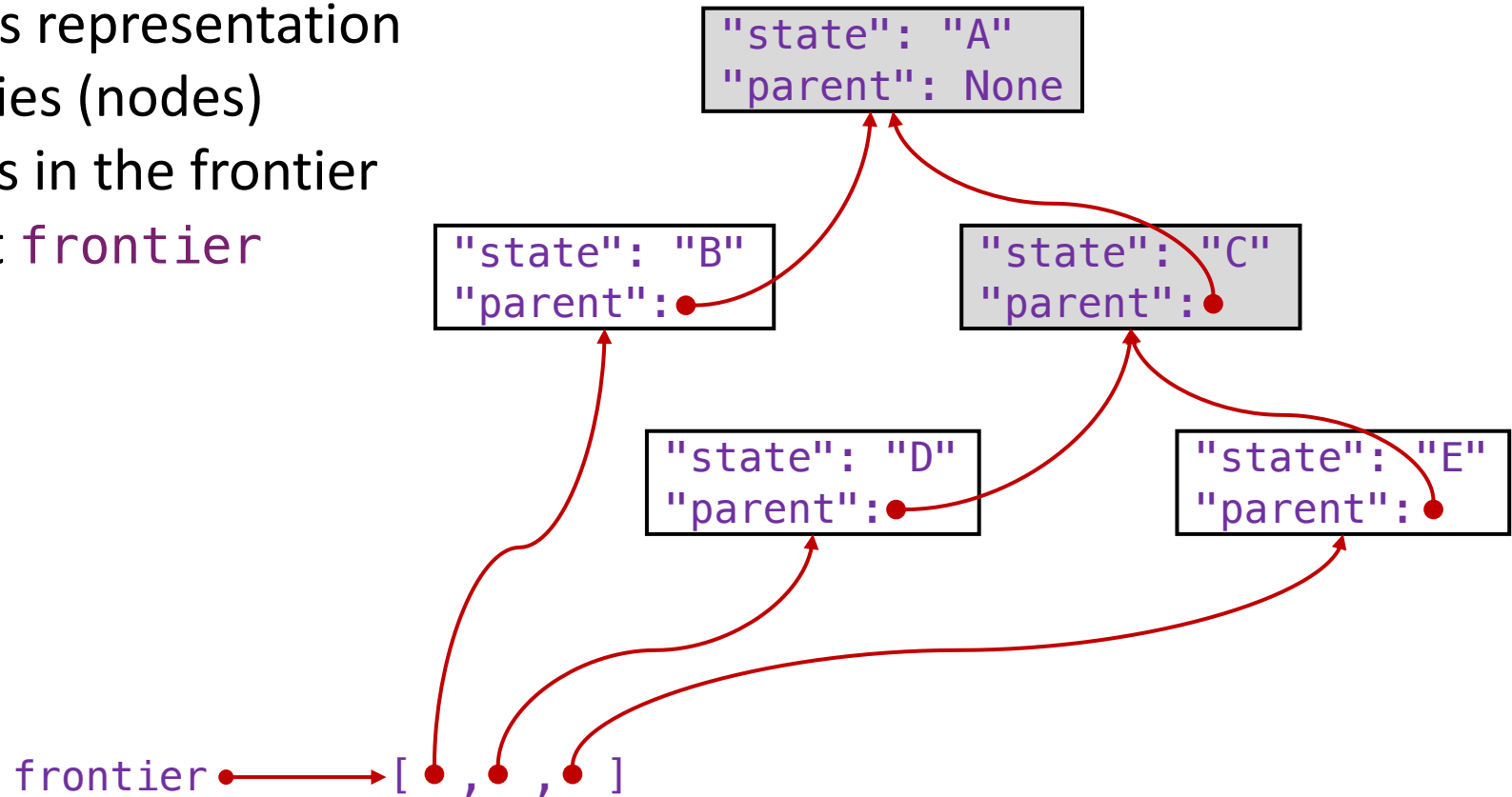
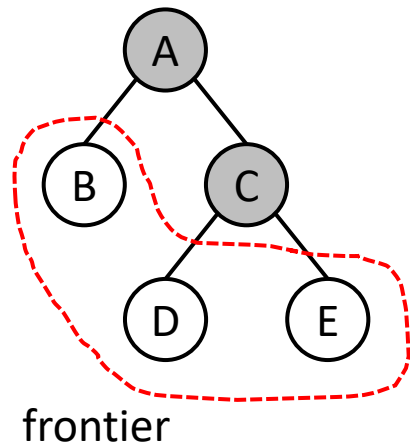
- ▶ What reference(s) to search tree elements should be **explicitly** kept?
 - a reference to the **root** node only?
 - but each node references only its **parent**... how to reach all the other nodes?
 - references to **each** node? (e.g., a **list** of references)
 - **redundant**: each node already includes a reference to its parent
 - how to find **leaf** nodes to choose the one to **expand** next?
- ▶ A **convenient** choice: storing references (e.g., a list) to **leaf** nodes only
 - leaf nodes correspond to **all** the **paths** represented by the current search tree
 - they allow the **whole paths** from the root to be easily **reconstructed** following backwards the references to parent nodes
 - this provides **direct access** to leaf nodes, to choose the one to expand next

Frontier of a search tree

- ▶ The set of leaf nodes of a search tree is called **frontier**, or **fringe**
 - they correspond to **all** the **paths** represented by the current tree
- ▶ In practice, a tree-search algorithm can keep in memory a **list** of references to the **current** frontier
 - the frontier is **initialised** as a list containing a reference to the **root** node
 - at each **iteration**
 - a leaf node is chosen for expansion and is **removed** from the frontier
 - its successors (new leaf nodes) are **added** to the frontier

Example

- ▶ A search tree and its representation as a set of dictionaries (nodes)
- ▶ References to nodes in the frontier are stored in the list `frontier`



Search strategies

Tree-search algorithm revisited

How to choose **which** leaf node to expand at each iteration?

- randomly?
- other criteria?
- does this choice make any difference?

```
function Tree-Search(problem) returns a solution, or failure
  generate the root node using the initial state of problem
  loop do
    if there are no leaf nodes
      return failure
    choose a leaf node n
    if n contains a goal state
      return the corresponding solution
    expand n
```

Search strategies

- ▶ Different criteria to choose the node to expand **do** make a **big** difference, leading to different **search strategies**
- ▶ Two main families
 - **uninformed** search
 - only the basic knowledge available for any search problem is used: initial state, goal state(s), operators and (only for some strategies) path cost
 - often not effective or not efficient enough for real-world problems
 - **informed** search
 - problem-specific knowledge is exploited (when available)
 - usually more efficient than uninformed search

Search strategies

Remember that

- **all** the search strategies we will consider use the **same** tree-search algorithm
- they differ **only** in the **criterion** for choosing the leaf node to expand next

```
function Tree-Search(problem) returns a solution, or failure
  generate the root node using the initial state of problem
  loop do
    if there are no leaf nodes
      return failure
    choose a leaf node n
    if n contains a goal state
      return the corresponding solution
    expand n
```

Uninformed search

Uninformed search strategies

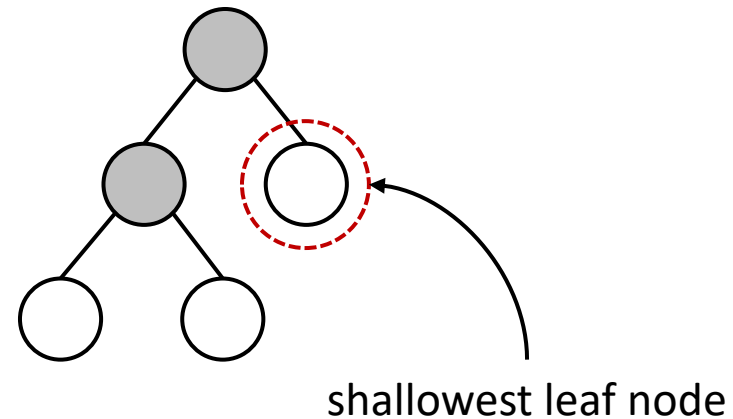
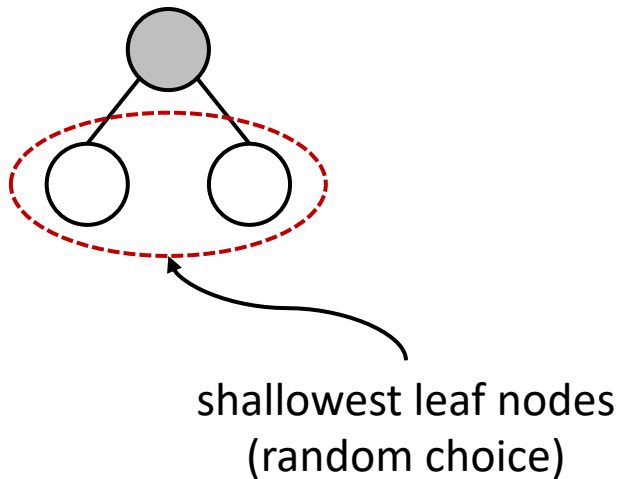
Two basic search strategies

- Breadth-first search (BFS)
- Depth-first search (DFS)

Breadth-first search

The **shallowest** leaf node is expanded

- in case of tie, one of the shallowest nodes is **randomly** chosen



Example: BFS for 8-puzzle

7	2	4
5		6
8	3	1

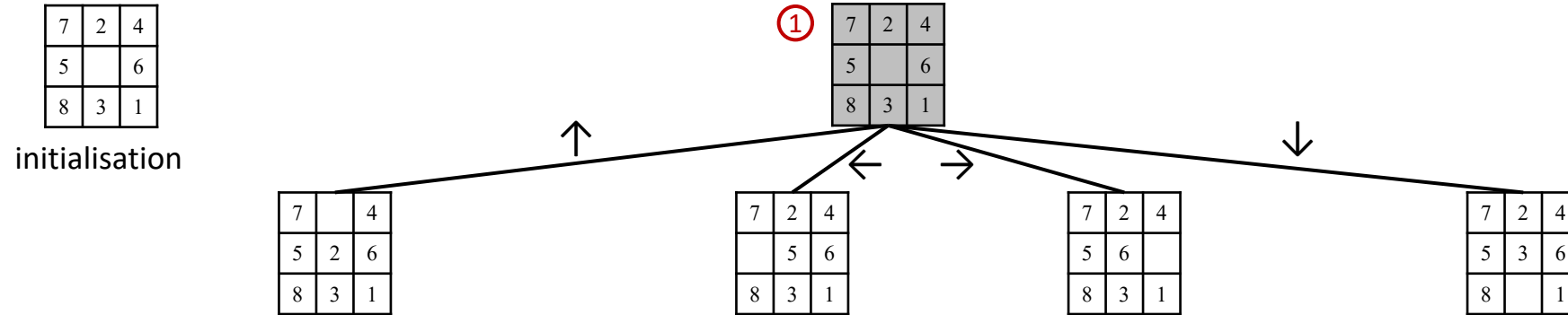
Start State

	1	2
3	4	5
6	7	8

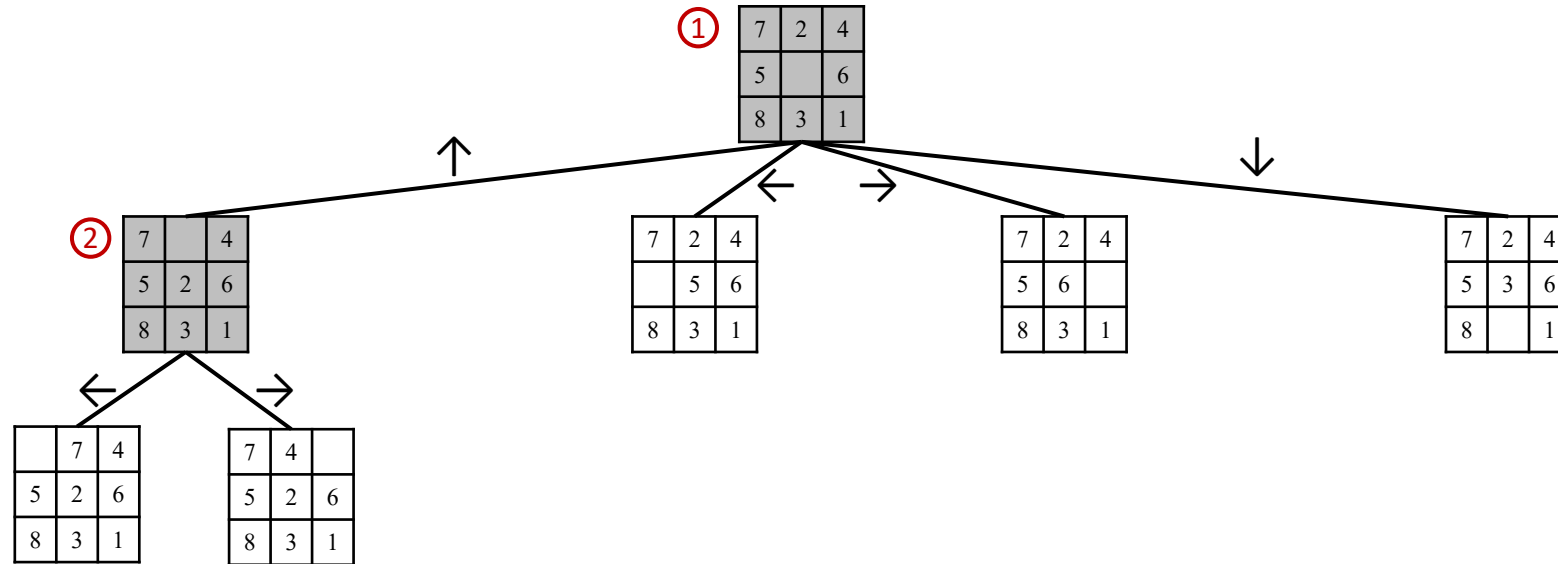
Goal State

- ▶ The first six expansion steps will be shown
- ▶ Cyclical paths will be avoided
- ▶ To make solutions comparable
 - newly generated nodes will be drawn left to right, following the operators' order $\uparrow \leftarrow \rightarrow \downarrow$
 - ties are broken not randomly, but according to the same order among operators

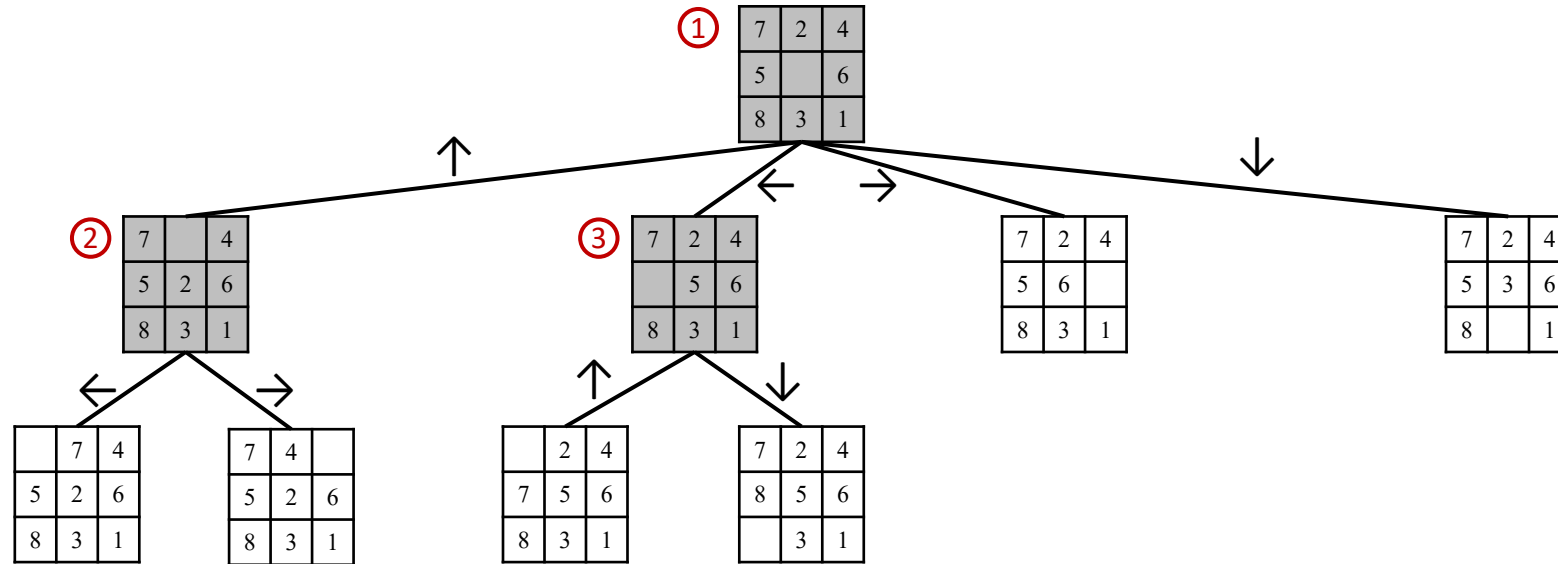
Example: BFS for 8-puzzle



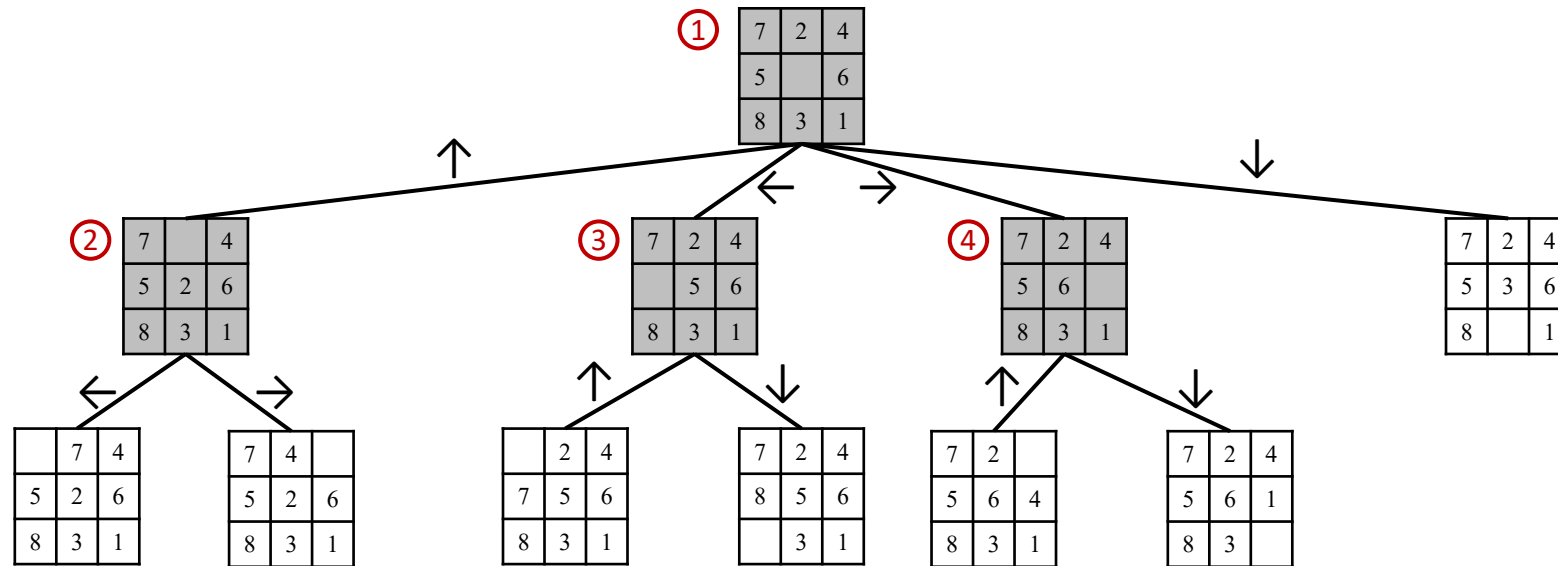
Example: BFS for 8-puzzle



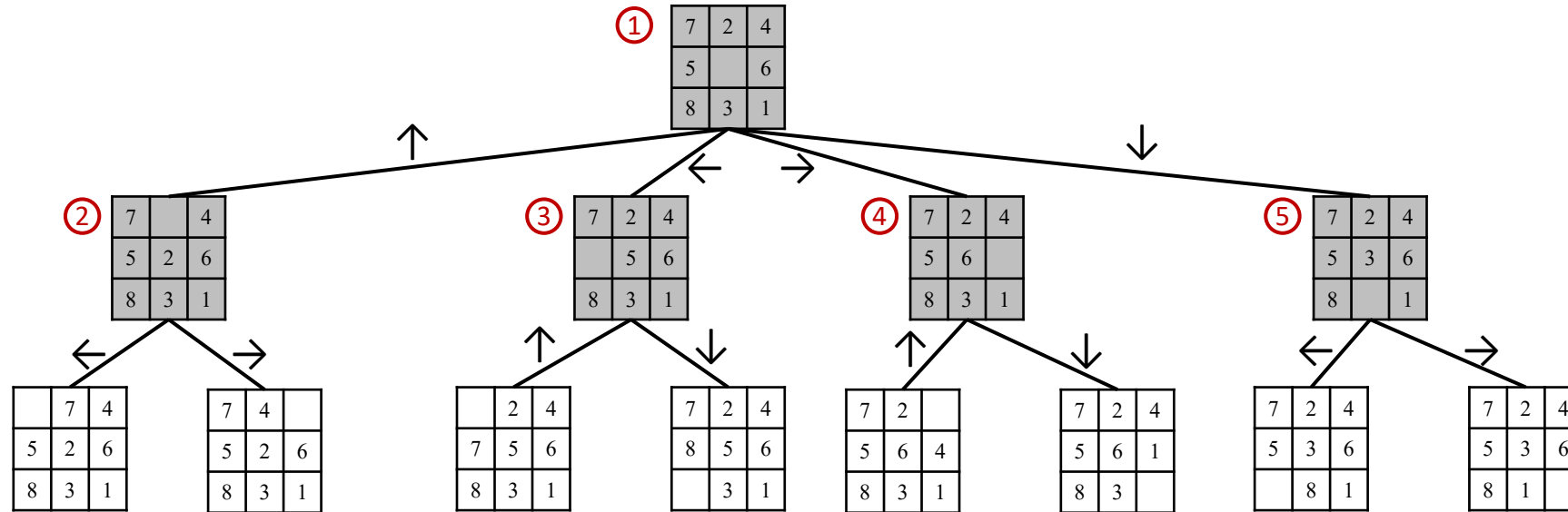
Example: BFS for 8-puzzle



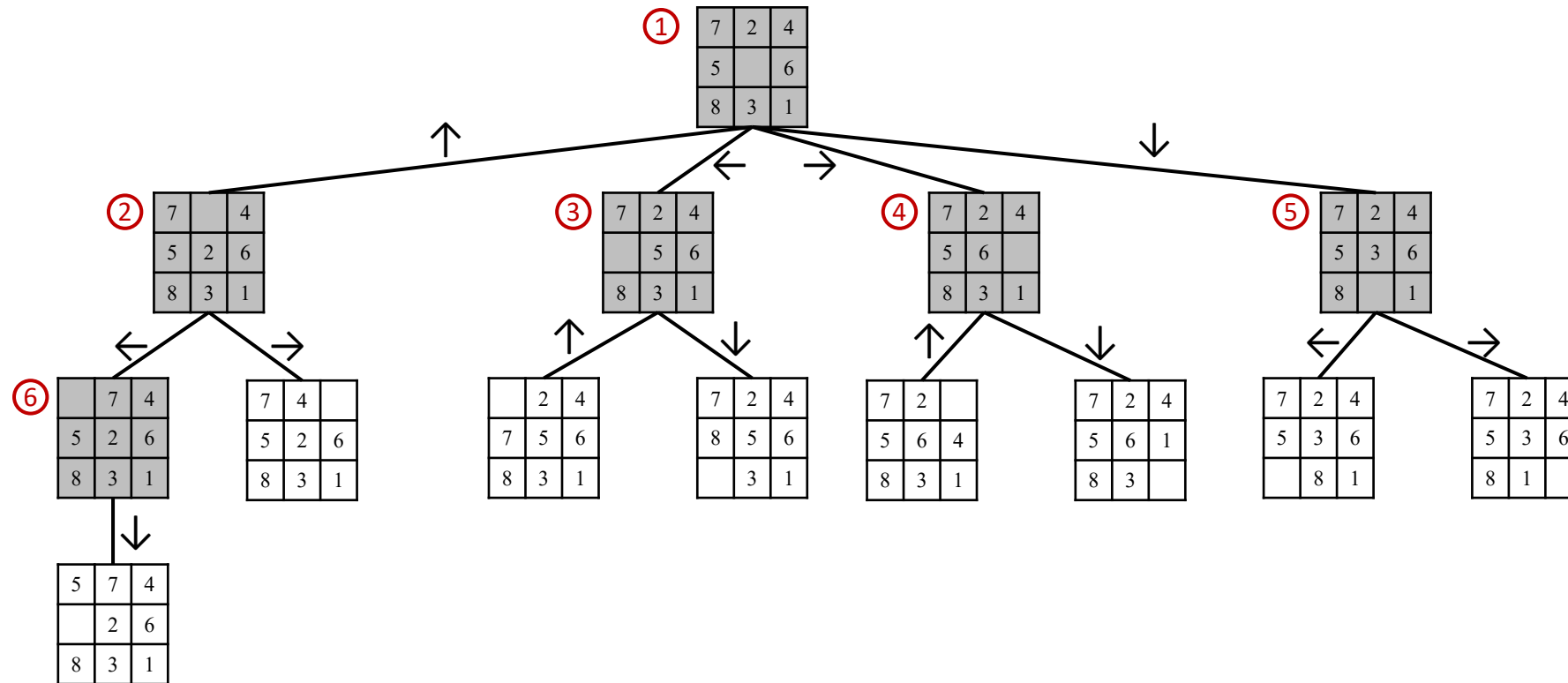
Example: BFS for 8-puzzle



Example: BFS for 8-puzzle



Example: BFS for 8-puzzle

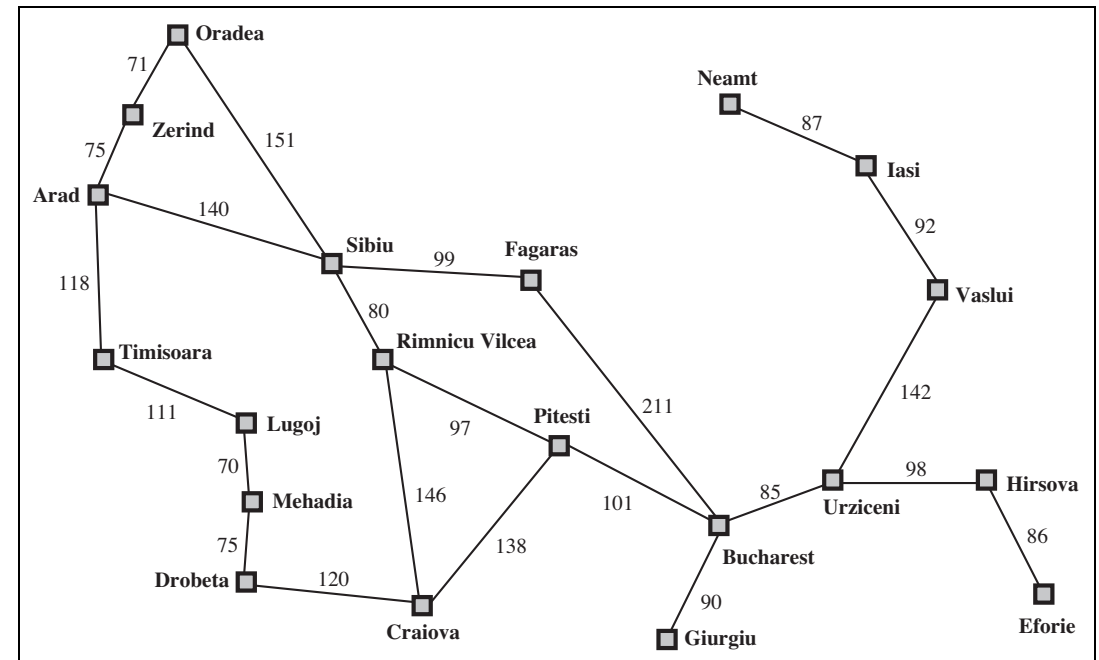


and so on...

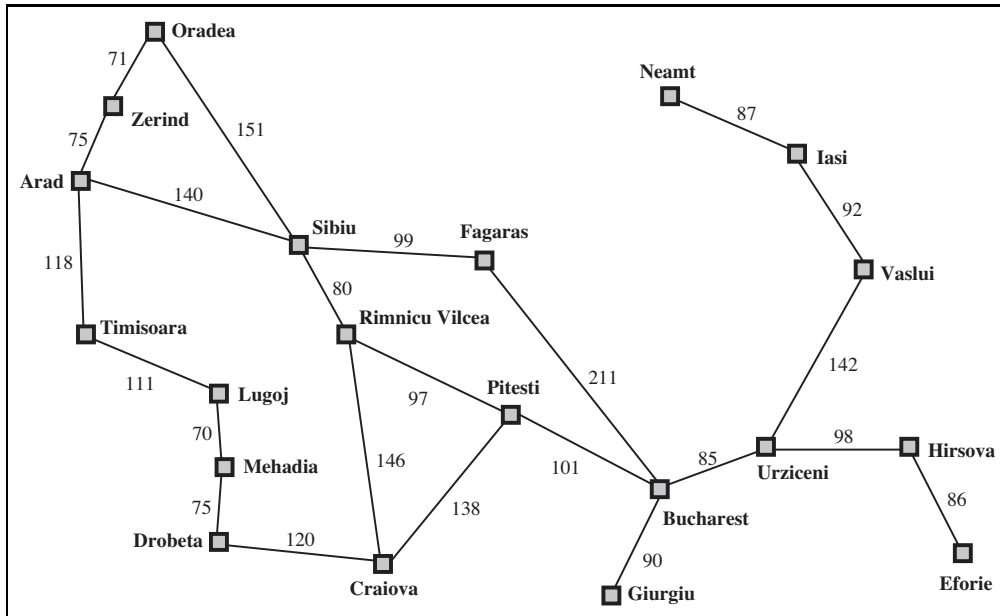
Exercise: BFS for route-finding

Find a route from Arad to Bucharest using BFS

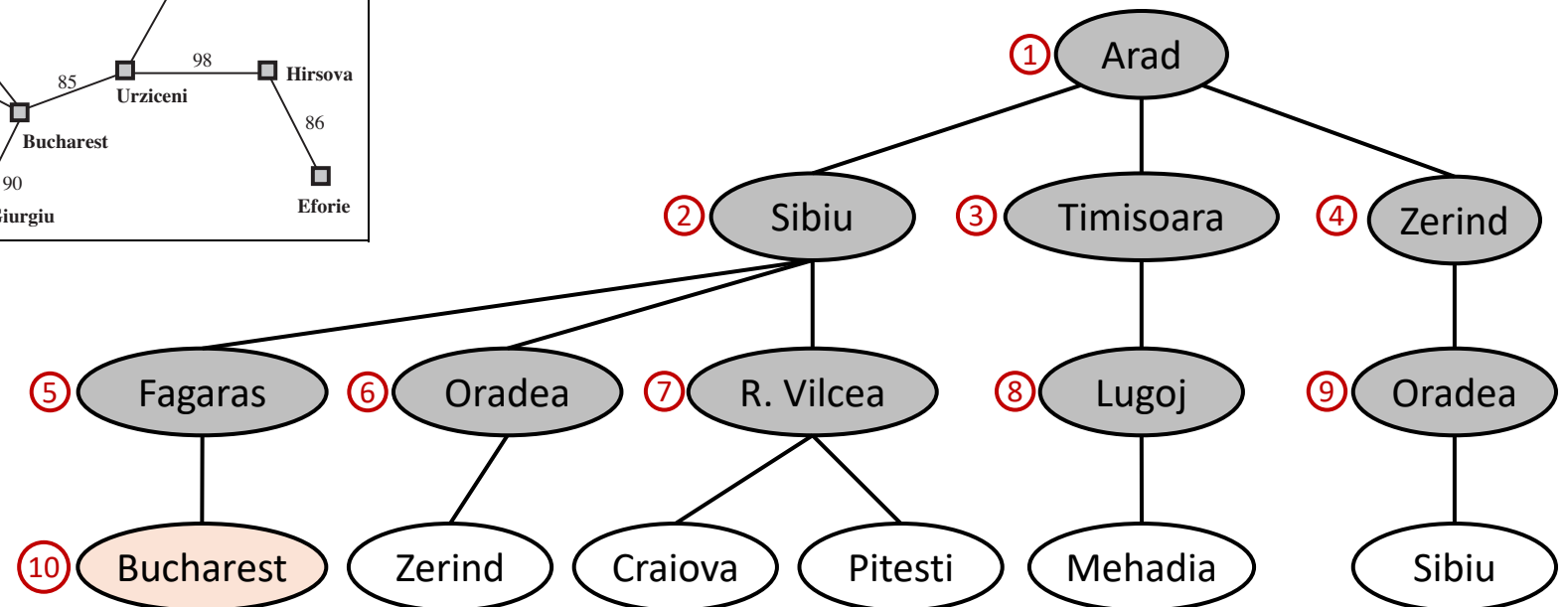
- avoid cyclical paths
- to compare your own solution with the one shown in the next slide, in case of tie choose the node to expand not randomly, but according to **alphabetical** order between city names



Solution



- Concise representation of the final search tree
- **Solution** found: (Arad, Sibiu, Fagaras, Bucharest)
- Note that a solution is found when a leaf node containing a goal state is chosen for **expansion** (step 10), not when it is **generated** (step 5)



Remarks about BFS

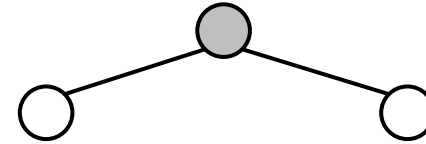
- ▶ **Whatever** tie breaker criterion is used, **all** the nodes at any given depth d are generated **before** the ones at depth $d + 1$
 - **all** the nodes at depth 1 are generated by expanding the root node
 - then **all** the nodes at depth 2 are generated by expanding (in any order) the nodes at depth 1
 - then **all** the nodes at depth 3 are generated by expanding (in any order) the nodes at depth 2
 - and so on...
- ▶ Does BFS find the **minimum-cost** solution?
 - look at the path found from Arad to Bucharest: are there shorter paths?
 - what about 8-puzzle?

Properties of search strategies: effectiveness

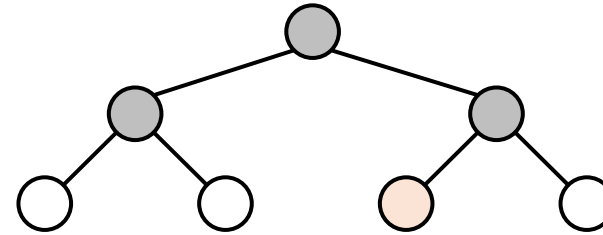
- ▶ **Effectiveness** refers to the **quality** of the solution found
- ▶ It can be evaluated along two dimensions
 - **completeness**: is a **solution** guaranteed to be found, if any?
 - **optimality**: is the **minimum-cost** solution guaranteed to be found?

Effectiveness of BFS

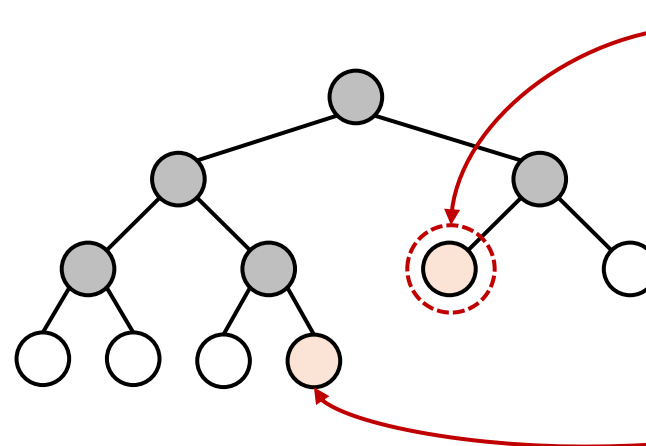
- ▶ Remember that
 - BFS generates all nodes at depth 1, then all nodes at depth 2, etc., **regardless** of path cost
 - a solution is found when the corresponding leaf node is selected for expansion
- ▶ It easily follows that
 - BFS is **complete**: it always finds a solution, if some exists
 - it finds the **shallowest** solution, not necessarily the minimum-cost one: thus it is **not optimal** (unless the path cost increases with the depth, e.g., in 8-puzzle) – how could it be optimal, given that it **disregards** the **path cost**?



after expanding the root...



after expanding nodes at depth 1...



a goal state is found when expanding nodes at depth 2, but other goal states (possibly cheaper) may exist at higher or identical depth...

Properties of search strategies: efficiency

- ▶ In computer science **efficiency** refers to the amount of **computing resources** required by an algorithm
- ▶ Formally: **computational complexity**
- ▶ Two main kinds of computing resources
 - CPU time: **time** complexity
 - memory: **space** complexity
- ▶ A **worst-case** evaluation is usually made
- ▶ However, CPU time and memory requirement depend on **contingent** factors (hardware, programming language, algorithm implementation, etc.) besides **intrinsic** properties of an algorithm...

Evaluation of computational complexity

- ▶ Intrinsic properties of an algorithm are evaluated as a function of **problem size** n (problem-specific definition)
 - time complexity: number of **elementary operations** of an algorithm
 - space complexity: number of stored **instances** of the main data structures
- ▶ **Asymptotic** evaluation as $n \rightarrow \infty$: *Big-O* notation, $\mathcal{O}(\dots)$
 - typical values: $\mathcal{O}(\log n)$, $\mathcal{O}(n)$, $\mathcal{O}(n^2)$, $\mathcal{O}(2^n)$, ...
- ▶ Example: selection sort
 - problem size: sequence length n
 - time complexity: number of pairwise comparisons, $\mathcal{O}(n^2)$
 - space complexity: sequence length, $\mathcal{O}(n)$

Search strategies: computational complexity

- ▶ Elementary operation: **generation** of **one** successor during the **expansion** step
 - **time** complexity: number of **generated** nodes until a solution is found
- ▶ Main data structure: a **node** of the search tree
 - **space** complexity: maximum **search tree size** (number of nodes)

```
function Tree-Search(problem) returns a solution, or failure
  generate the root node using the initial state of problem
  loop do
    if there are no leaf nodes
      return failure
    choose a leaf node n
    if n contains a goal state
      return the corresponding solution
    expand n
```

Search strategies: computational complexity

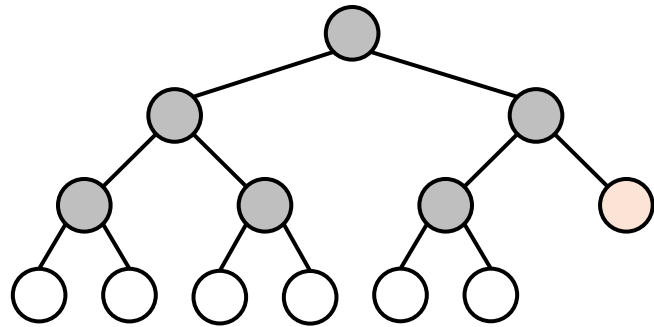
- ▶ However, the number of generated nodes and the maximum search tree size depend on
 - the **branching factor** b : **maximum** number of successors of any node
 - the depth d of the **shallowest** goal node
 - the **maximum** length m of any path in the state space (avoiding cyclical paths)
- ▶ To make an analytical evaluation feasible, simplifying assumptions on b , d and m can be made, depending on the specific search strategy
 - one common assumption: each node has **exactly** b successors (**constant** branching factor)

BFS: time complexity

- ▶ To evaluate the worst-case number of generated nodes, consider that
 - BFS generates **all** nodes at depth 1, then **all** nodes at depth 2, etc.
 - a solution is found when a goal node is **selected** for expansion
 - BFS finds the **shallowest** solution (depth d)
- ▶ Worst case: the goal node is the **last** one selected for expansion at depth d

BFS: time complexity

- ▶ **Binary** search tree: constant branching factor $b = 2$
- ▶ Depth of the shallowest solution: $d = 2$
- ▶ Number of generated nodes at each depth:



depth 0: $1 = b^0$ node

depth 1: $2 = b^1$ nodes

depth 2: $4 = b^2$ nodes

depth 3: $6 = b^3 - b = b^{d+1} - b$ nodes (worst case: the **last** node selected for expansion at depth d contains a goal state)

- ▶ Total number of generated nodes: **exponential** in the shallowest solution depth

$$1 + b^1 + b^2 + \dots + b^{d+1} - b = \mathcal{O}(b^{d+1})$$

BFS: space complexity

- ▶ Worst case for the maximum search tree size
 - **every** generated node must remain in memory
 - this happens when each node selected for expansion has at least one successor (avoiding cyclical paths)
- ▶ It easily follows that the worst-case maximum search tree size equals the total number of generated nodes:

$$1 + b^1 + b^2 + \dots + b^{d+1} - b = \mathcal{O}(b^{d+1})$$

An example

- ▶ Time and space complexity are both **exponential** in the shallowest solution depth: $\mathcal{O}(b^{d+1})$... not a good news...
- ▶ To get an insight into what this means, consider a problem with
 - branching factor $b = 10$
 - CPU time for generating one node: 10^{-4} s
 - storage required for a single node: 1,000 bytes
- ▶ The following table shows that BFS is not a feasible strategy for real-world problems...

An example

Worst-case search tree size, CPU time and memory requirement as a function of the shallowest solution depth d , up to $d = 14$:

d	n. of generated nodes	CPU time	memory
2	1,101	0.11 seconds	1 megabyte
4	111,101	11 seconds	106 megabytes
6	10^7	19 minutes	10 gigabytes
8	10^9	31 hours	1 terabyte
10	10^{11}	129 days	101 terabytes
12	10^{13}	35 years	10 petabytes
14	10^{15}	3,523 years	1 exabyte

- time and space complexity become **excessive** even for small values of d
- memory requirement is a **bigger** problem than processing time

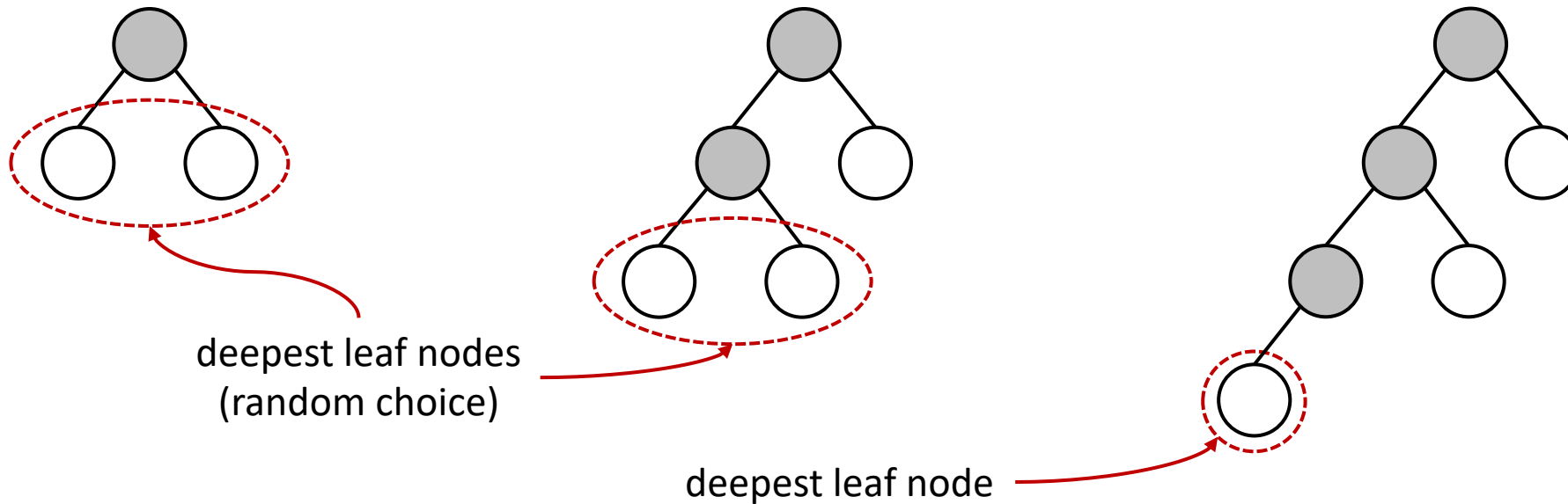
Example: 8-puzzle

- ▶ You may think 8-puzzle is a toy-problem, not a real-world one...
- ▶ Let us have a closer look
 - state space size for solvable problem instances: $9!/2 = 181,440$
 - **average** shallowest solution's depth (over all problem instances): $d \cong 22$
 - **average** branching factor (over all $9!$ states): $b \cong 3$
- ▶ Worst-case time and space complexity using BFS, for **average** d and b : $\mathcal{O}(b^{d+1})$, with $b^{d+1} = 3^{23} \cong 9.4 \times 10^{10} \dots$
 - about **94 billion nodes** have to be generated and stored in memory...
 - if node generation takes 10^{-6} s, processing time is $\cong 9.4 \times 10^4$ s ($\cong$ **26 hours**)
 - if the storage for a node is 16 bytes, about **1.37 terabytes** are required...

Depth-first search

DFS expands at each iteration the **deepest** leaf node

- in case of tie, one of the deepest nodes is **randomly** chosen



Example: DFS for 8-puzzle

7	2	4
5		6
8	3	1

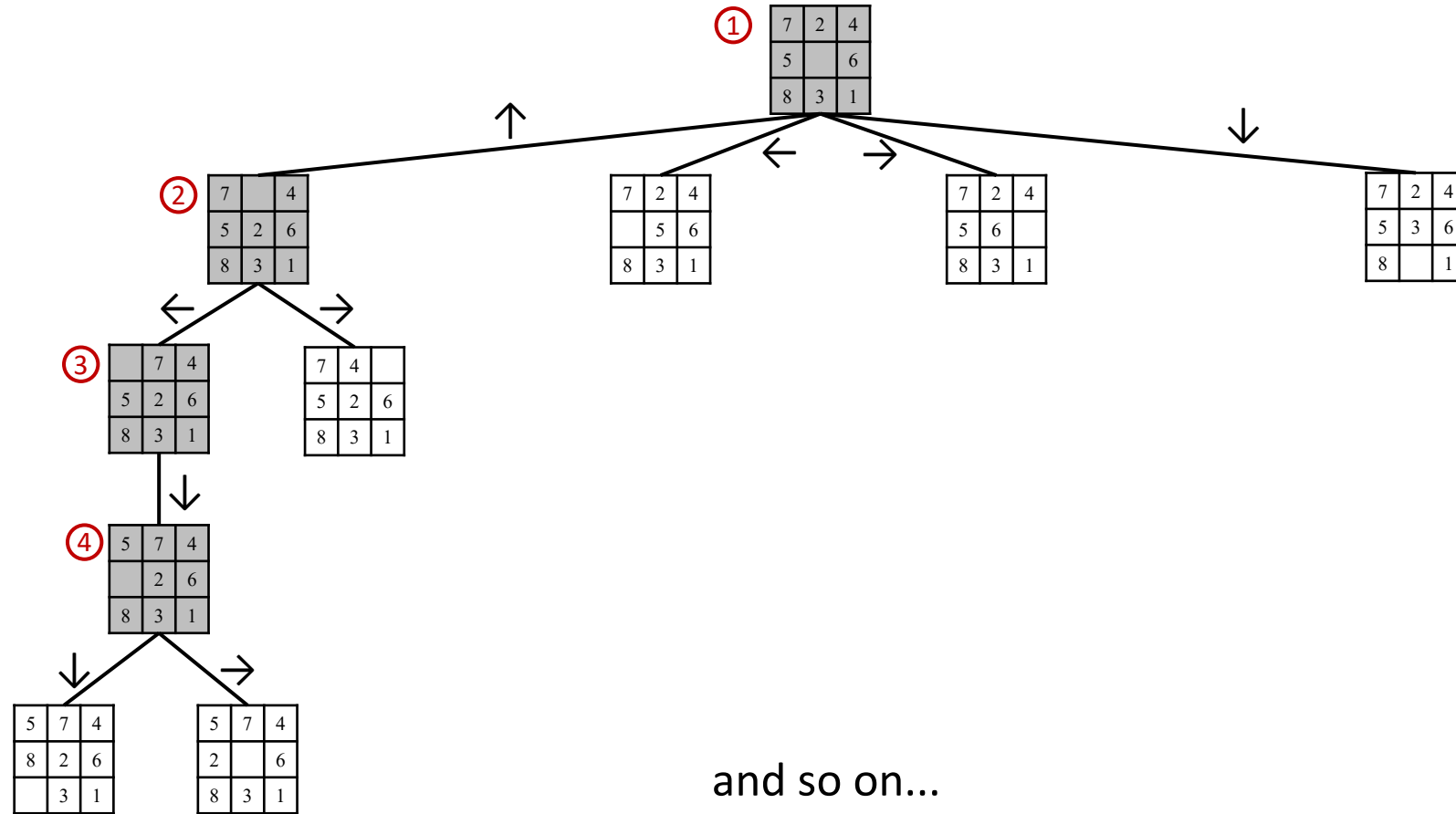
Start State

	1	2
3	4	5
6	7	8

Goal State

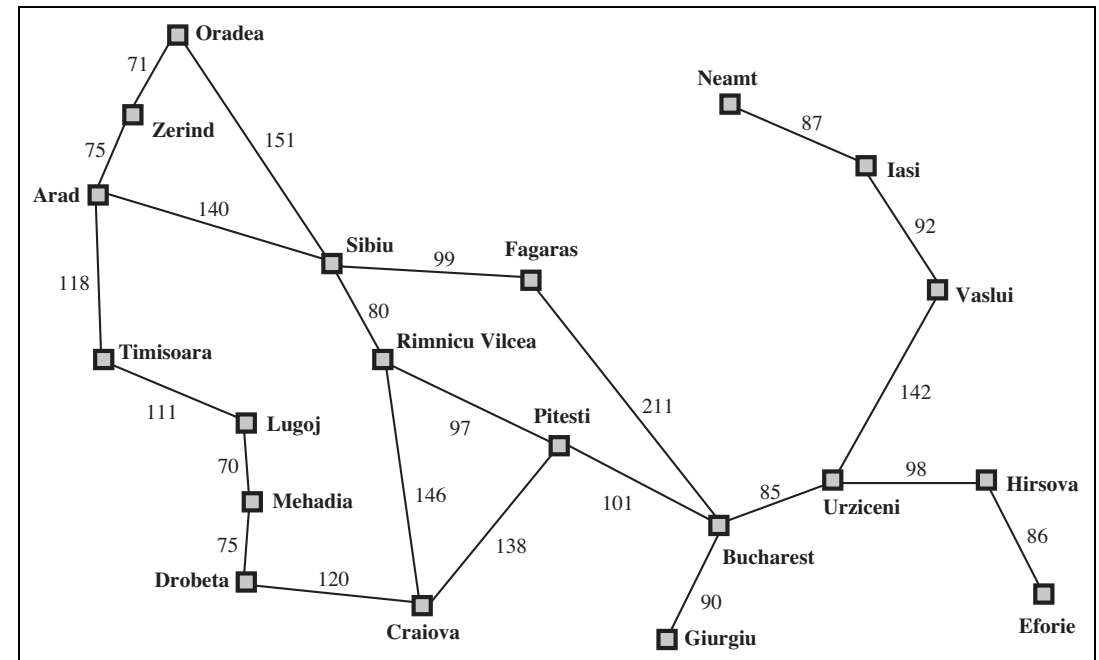
- ▶ As in the example on BFS
 - cyclical paths will be avoided
 - newly generated nodes will be drawn left to right, following the operators' order $\uparrow \leftarrow \rightarrow \downarrow$
 - ties are broken not randomly, but following the same order among the operators
- ▶ Only the first four steps are shown

Example: DFS for 8-puzzle

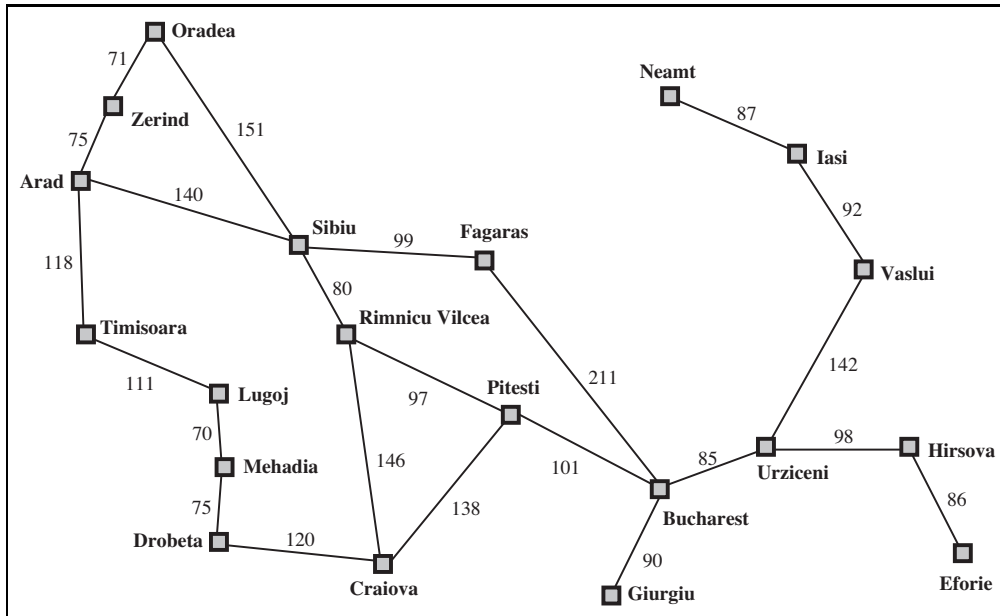


Exercise: DFS for route-finding

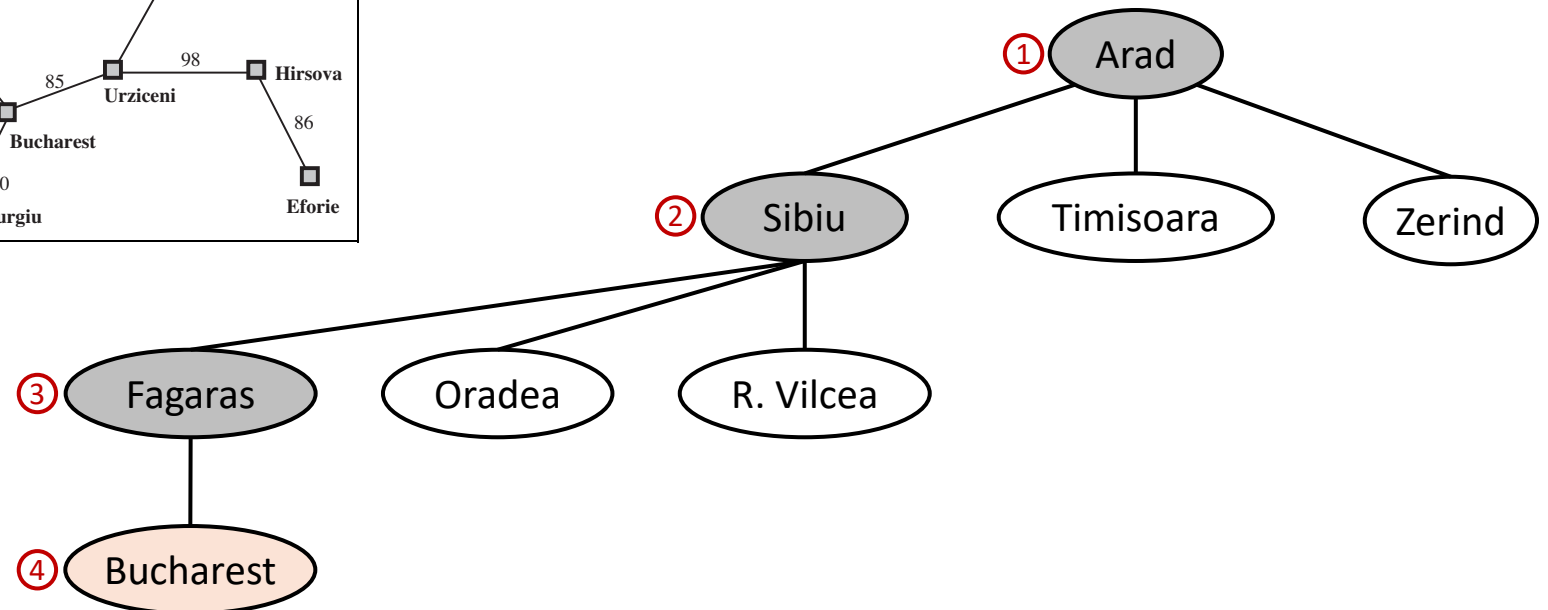
- ▶ Use DFS to find a route from Arad to Bucharest
- ▶ As in the case of BFS
 - avoid cyclical paths
 - in case of tie choose the node to expand not randomly, but according to **alphabetical** order between city names



Solution

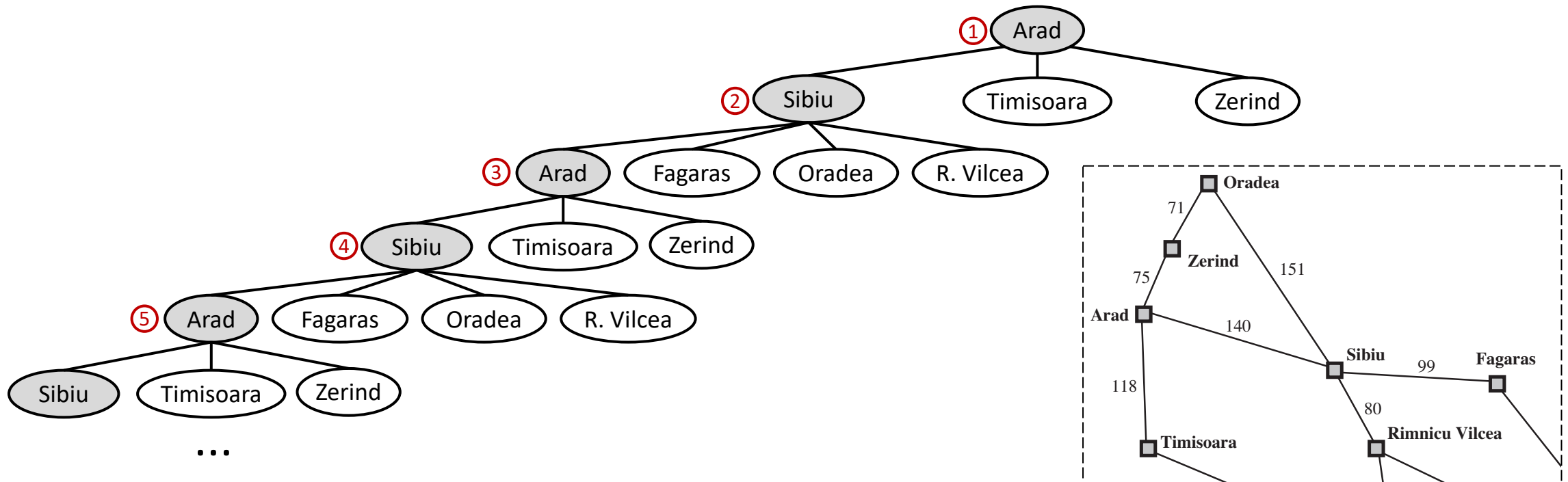


- ▶ **Solution** found: (Arad, Sibiu, Fagaras, Bucharest) (same as BFS, but just by chance)
- ▶ Note again that the solution is found when the corresponding node is selected for expansion (step 4), not when it is generated (step 3)



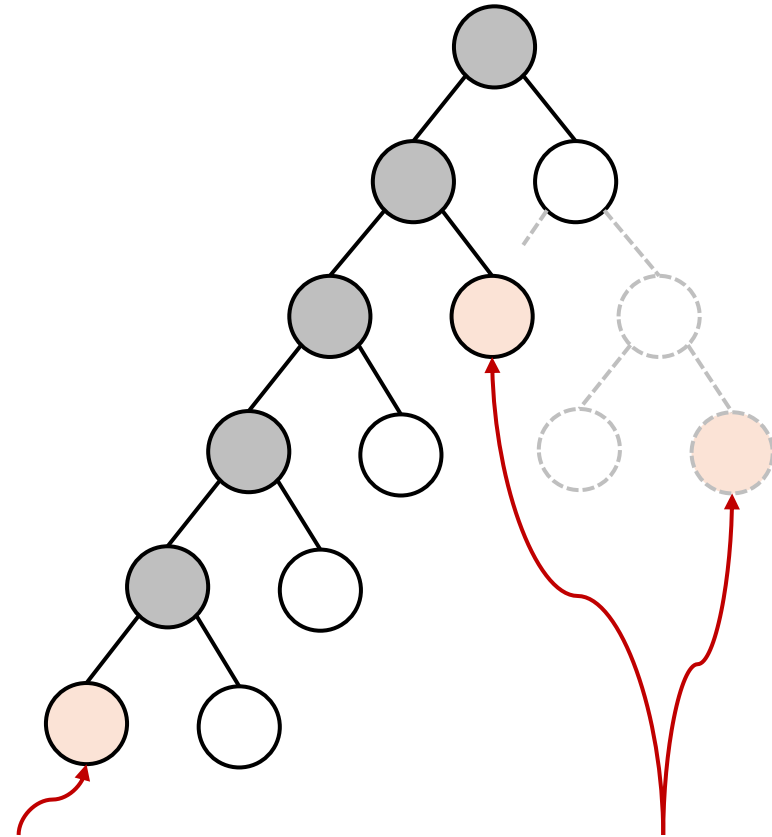
Avoiding cyclical paths in DFS

- ▶ Avoiding cyclical paths is **crucial** for DFS
- ▶ Otherwise it could get stuck in **infinite** paths
 - an example on route-finding:



Effectiveness of DFS

- ▶ DFS **fully** explores a **single** path at a time, regardless of path cost, until a solution is found or (if cyclical paths are avoided) until a node has no successors
- ▶ It easily follows that DFS is
 - **complete**, provided that cyclical paths are avoided
 - **not optimal**: lower-cost solutions may exist in paths not yet explored (same issue as BFS: also DFS disregards the path cost...)



a goal state is found
when this node is
selected for expansion...

other goal states, possibly cheaper, may exist in paths not yet explored...

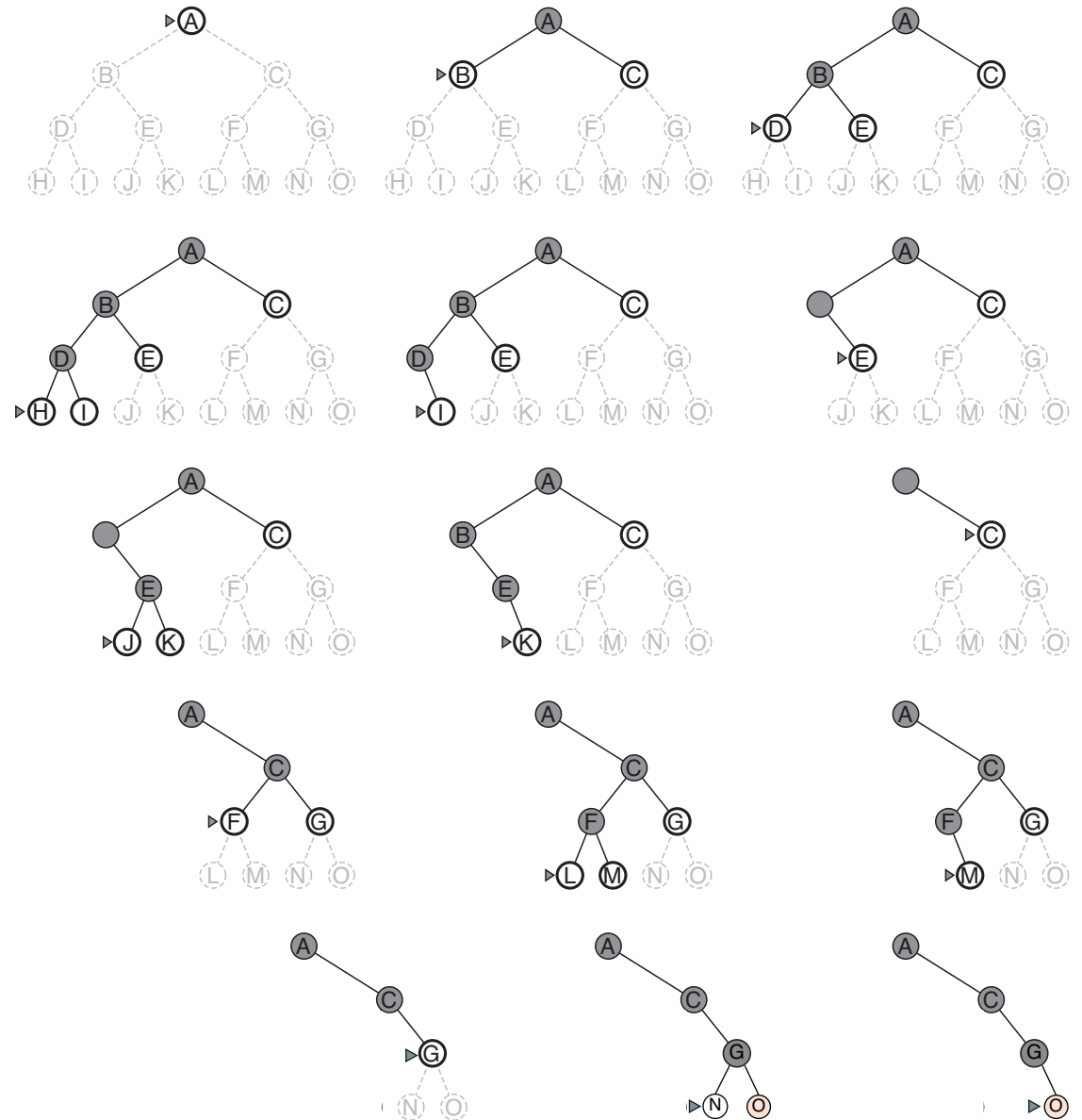
DFS: space complexity

- ▶ Worst-case, maximum search tree size
 - all paths have identical length m
 - a **single** goal node exists, at depth $d = m$, in the **last** explored path
- ▶ However, nodes with no successors (e.g., when all the corresponding states are already in the same path) can be **removed** from memory: see the example in the next slide
- ▶ Therefore the maximum search tree size grows just **linearly** with mb :

$$1 + \underbrace{b + b + \cdots + b}_m = 1 + mb = \mathcal{O}(mb)$$

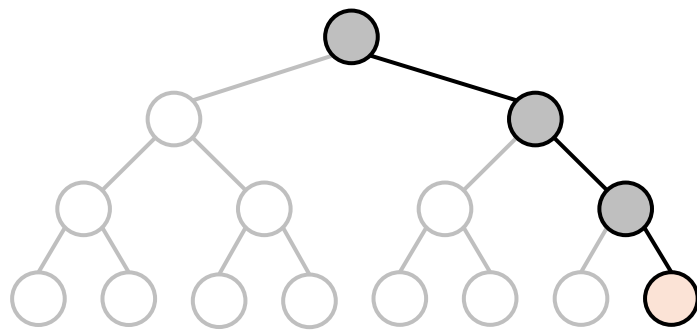
Example

- ▶ A search tree with
 - branching factor $b = 2$
 - same length $m = 3$ for all paths
 - shallowest solution (goal state 'O') in the **last** explored path, at depth $d = m = 3$
- ▶ Light grey: nodes not yet generated
- ▶ The triangle marks the node to expand next
- ▶ Only the root node and at most b nodes for each depth level (up to depth m) must be kept in memory, i.e., at most $1 + mb$ nodes



DFS: time complexity

- ▶ Worst-case, maximum search tree size: same assumptions as for space complexity
- ▶ It follows that **all** the nodes must be generated before a solution is found...
- ▶ An example (light grey: nodes already generated and removed from memory)



depth 0: $1 = b^0$ node

depth 1: $2 = b^1$ nodes

depth 2: $4 = b^2$ nodes

depth 3: $8 = b^3 = b^m$ nodes (the **last** node selected for expansion at depth $d = m = 3$ contains a goal state)

- ▶ Total number of generated nodes:

$$1 + b^1 + b^2 + \dots + b^m = \mathcal{O}(b^m)$$

- ▶ Time complexity remains **exponential** in the shallowest solution depth, as that of BFS...

Other uninformed search strategies

- ▶ **Uniform-cost** search: achieving optimality by taking into account the path cost when choosing the node to expand next
- ▶ Variants of DFS to reduce time complexity, while keeping space complexity negligible
 - **depth-limited** search
 - **iterative deepening** depth-first search
- ▶ **Bidirectional** search: trying to make computational complexity practically manageable, though still exponential

Uniform-cost search

The leaf node with the **lowest path cost** is chosen for expansion

- ties are **randomly** broken

► Effectiveness

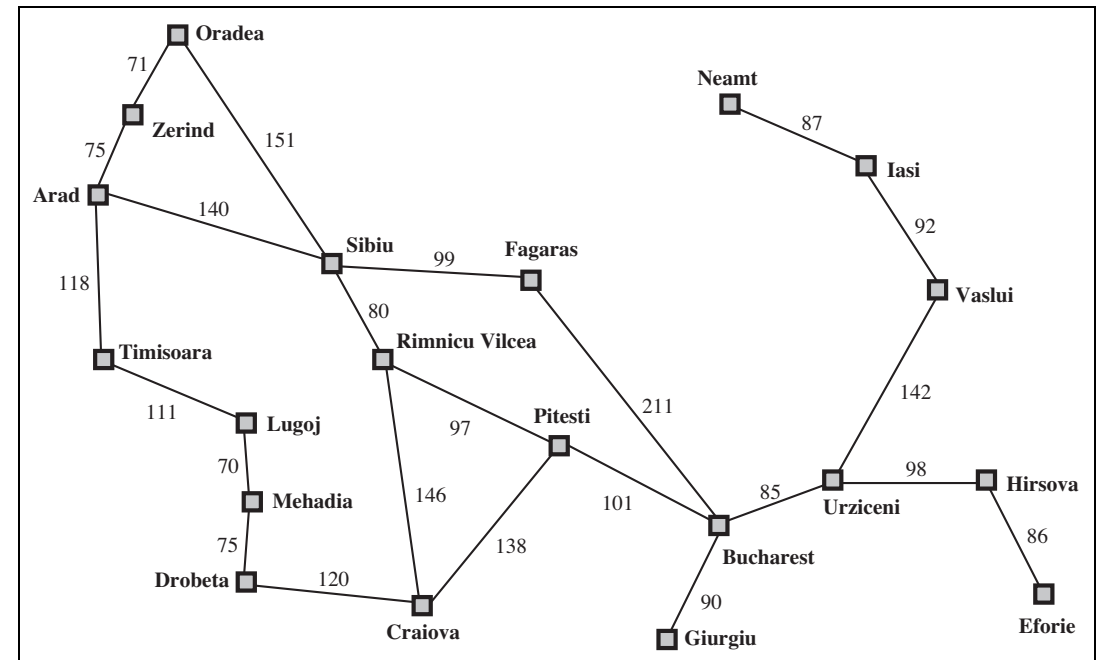
- it is **complete**, unless operators with zero step cost exist
- it is **optimal** (the proof is left as an **exercise**), contrary to BFS and DFS

► Efficiency

- **exponential** worst-case time and space **complexity**: $\mathcal{O}(b^{\lceil C^*/\epsilon \rceil})$, where
 - C^* : cost of the optimal solution
 - ϵ : minimum step cost
 - $\lceil C^*/\epsilon \rceil$: upper bound of the optimal solution depth

Exercise

- ▶ Use uniform-cost search to find a route from Arad to Bucharest
 - boring, but useful...
- ▶ If you do not make any error...
 - you will find the solution: (Arad, Sibiu, Rimnicu Vilcea, Pitesti, Bucharest), whose path cost is 418 km
 - it is easy to see it is the **optimal** solution, though **not** the shallowest one...
 - ...therefore it **cannot** be found by BFS (DFS can find it, but only by chance)



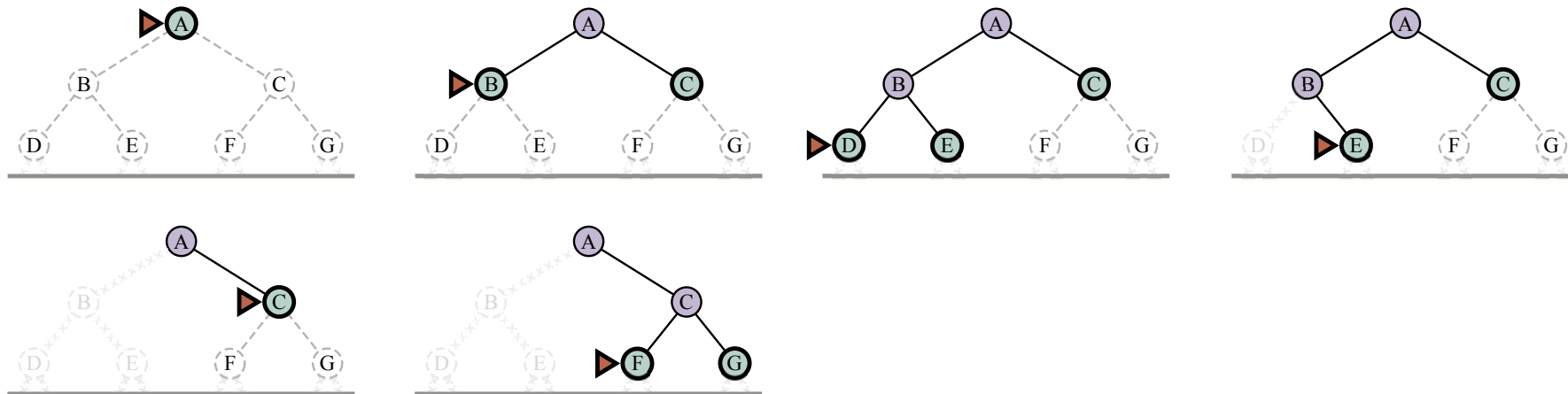
Depth-limited search

Running DFS with a given **depth limit** ℓ : every node at depth ℓ is treated as having no successors

- reduces **time complexity** from $\mathcal{O}(b^m)$ to $\mathcal{O}(b^\ell)$:
useful if one knows that the number of steps between any two states in the state space is at most ℓ , and $\ell < m$ (however, ℓ is usually **unknown** in real-world problems...)
- negligible **space complexity**, analogously to DFS: $\mathcal{O}(\ell b)$
- **complete**, if $\ell \geq d$ (but also d may be **unknown** in real-world problems...)
- **not optimal**, like DFS

Example

- ▶ A search tree with
 - branching factor $b = 2$
 - shallowest solution (goal state 'F') at depth $d = 2$
 - maximum depth $m > 2$
 - depth limit $\ell = d = 2$
- ▶ Dotted nodes are **not** yet generated
- ▶ Shaded nodes have already been generated and **removed** from memory
- ▶ The triangle marks the node to expand next

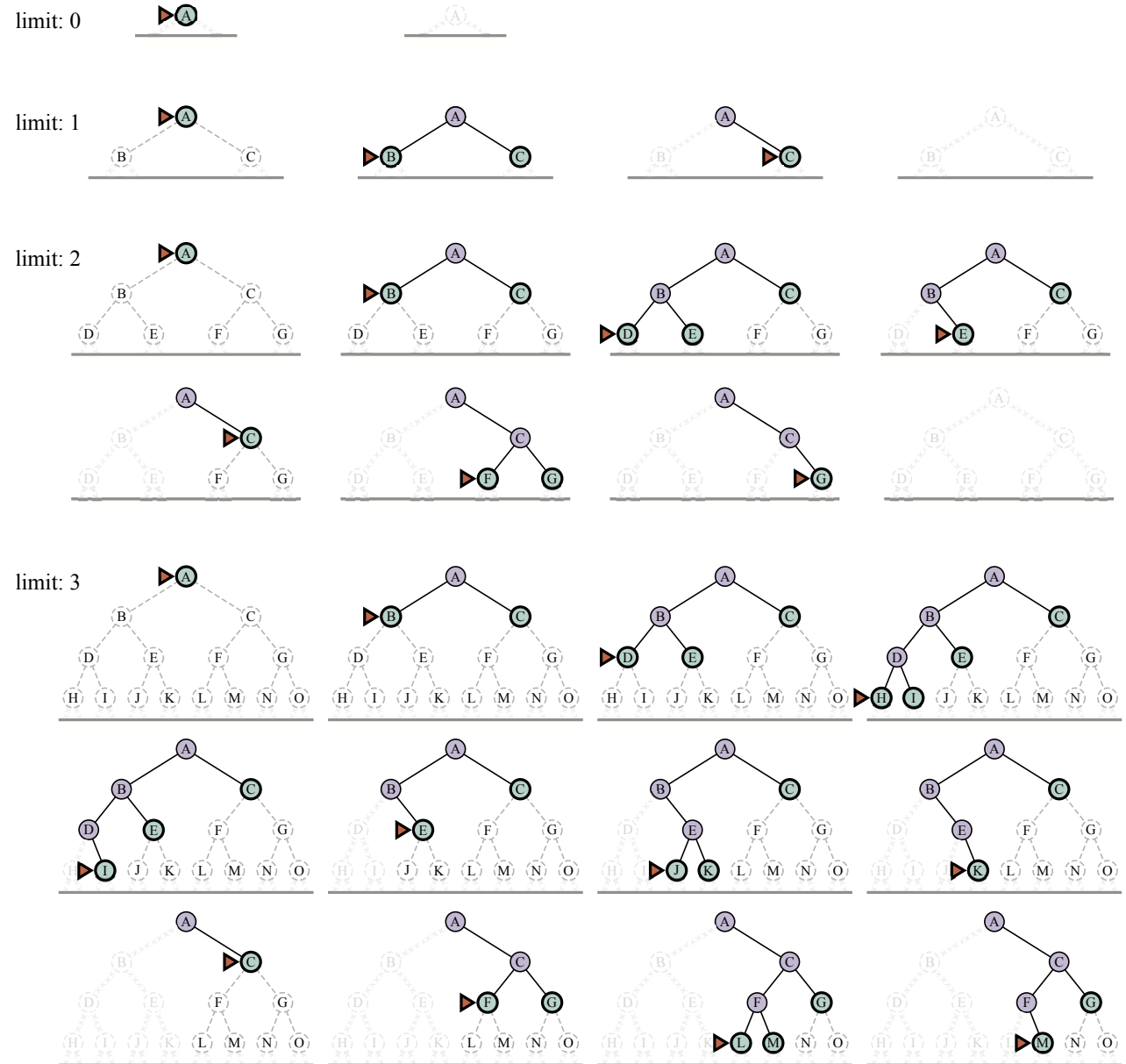


Iterative deepening depth-first search

- ▶ A specific instance of the general **iterative deepening search** strategy, combined with DFS (can be used with other strategies)
 - running depth-limited search for $\ell = 0, 1, 2, \dots$ until a solution is found, which occurs when ℓ reaches d (depth of the shallowest solution – usually **unknown**)
- ▶ Effectiveness: same as DFS
- ▶ Efficiency
 - **space** complexity: same as DFS, $\mathcal{O}(db)$
 - **time** complexity: wasted time for each run with $\ell < d$ is **negligible**: nodes at depth d are generated only once (when $\ell = d$), so time complexity remains $\mathcal{O}(b^d)$
- ▶ IDS is therefore the **preferred** uninformed search strategy for
 - large state spaces
 - unknown solution depth

Example

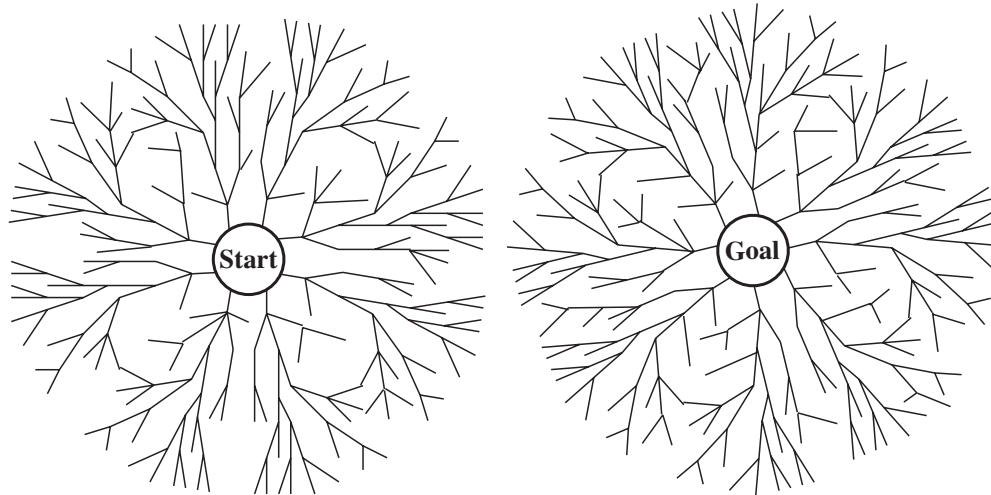
- ▶ A search tree with
 - branching factor $b = 2$
 - shallowest solution (goal state 'M') at depth $d = 3$
 - maximum depth $m > 3$
- ▶ The solution is found when the depth limit is $\ell = 3$



Bidirectional search

Basic idea: running two **simultaneous** searches (two search trees)

- one **forward** from the **initial** state, one **backward** from the **goal** state...
- ...**alternating** one expansion step on each search tree...
- ...until they **meet** (i.e., two nodes in their **frontiers** contain the **same** state)

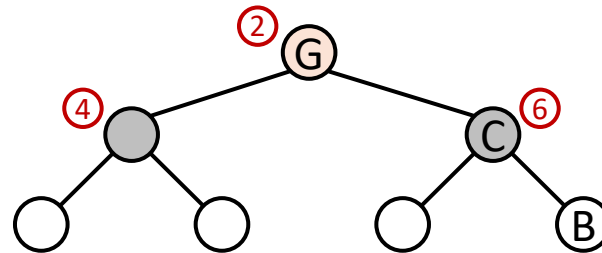
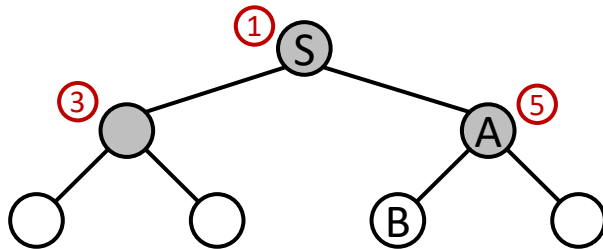
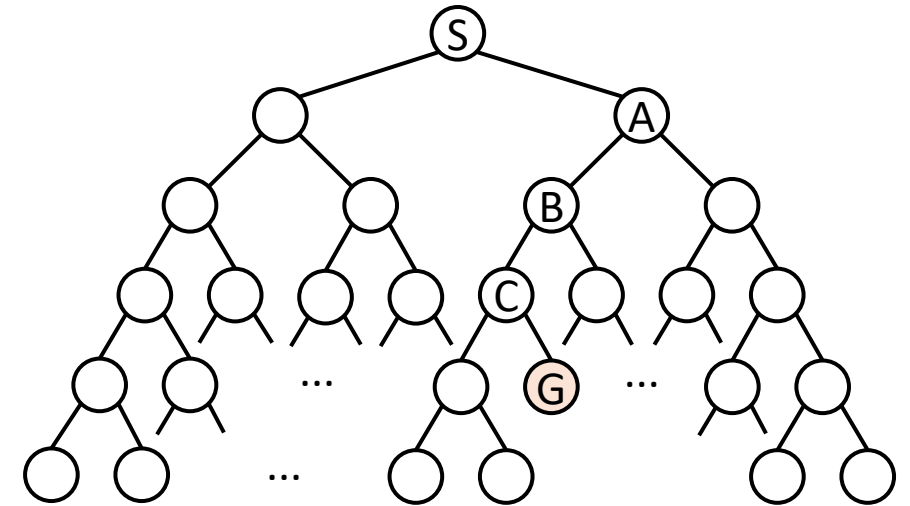


Bidirectional search

- ▶ Rationale: computational complexity can be significantly reduced
 - if BFS is used for both searches, the shallowest solution is found when one or both trees have depth $\lceil d/2 \rceil$
 - **time** and **space** complexity reduce from $\mathcal{O}(b^{d+1})$ to $\mathcal{O}(b^{\lceil d/2 \rceil + 1})$
(but is not guaranteed to be enough...)
- ▶ Effectiveness (using BFS): **complete** but **not optimal**

Bidirectional search: an example

- ▶ Right: a binary search tree with shallowest goal state 'G' at depth 4 (path: S, A, B, C, G)
- ▶ Bottom: the search trees constructed by bidirectional search, until a common state ('B') appears in both frontiers
- ▶ Tree depth is reduced from 5 to 2



Bidirectional search vs BFS

Consider the previous example on BFS

- if the depth of the shallowest solution is $d = 12$, BFS is **infeasible**
- bidirectional search grows two search trees up to a depth of 6, requiring "just" $2 \times 10^7 = 20$ gigabytes and $2 \times 19 = 38$ minutes
- however, if the depth of the shallowest solution is, e.g., $d = 24$...

d	n. of generated nodes	processing time	memory
2	1,101	0.11 seconds	1 megabyte
4	111,101	11 seconds	106 megabytes
6	10^7	19 minutes	10 gigabytes
8	10^9	31 hours	1 terabyte
10	10^{11}	129 days	101 terabytes
12	10^{13}	35 years	10 petabytes
14	10^{15}	3,523 years	1 exabyte

Issues of bidirectional search

- ▶ How to deal with **non-reversible** operators?
 - e.g., one-way roads in route-finding
- ▶ How to deal with **more than one** goal state?
 - for some problems goal states can be **exhaustively** listed
 - for others, they can only be **described**, e.g., checkmate in chess...

Implementation examples (part II)

Updating the search tree frontier

- ▶ We have seen that the search tree can be implemented through a list of references to the current leaf nodes (frontier)
- ▶ To efficiently find the node to expand next, the frontier can be implemented as a **sorted list**, such that the next node is always at the **top** of the list
 - BFS: the depth of newly generated nodes is **not lower** than the one of the current frontier: they are appended at the **end** of the list
 - DFS: the depth of newly generated nodes is **higher** than the one of the current frontier: they are inserted at the **top** of the list
 - UCS: newly generated nodes are inserted for **increasing path cost**

Implementing different search strategies

- ▶ Each "basic" search strategy (e.g., BFS, DFS, UCS) can be implemented as a **function** that adds a single, newly generated node to the frontier
- ▶ A reference to the function corresponding to the desired strategy (i.e., the function **name**) can then be passed as the second argument to the **Tree-search** function, which will use it to update the frontier after each expansion step

```
function Tree-Search(problem, strategy) returns a solution, or failure
  frontier ← [generate the root node using the initial state of problem]
  loop do
    if frontier is empty
      return failure
    remove the first leaf node n from frontier
    if n contains a goal state
      return the corresponding solution
    expand n and update frontier according to strategy
```

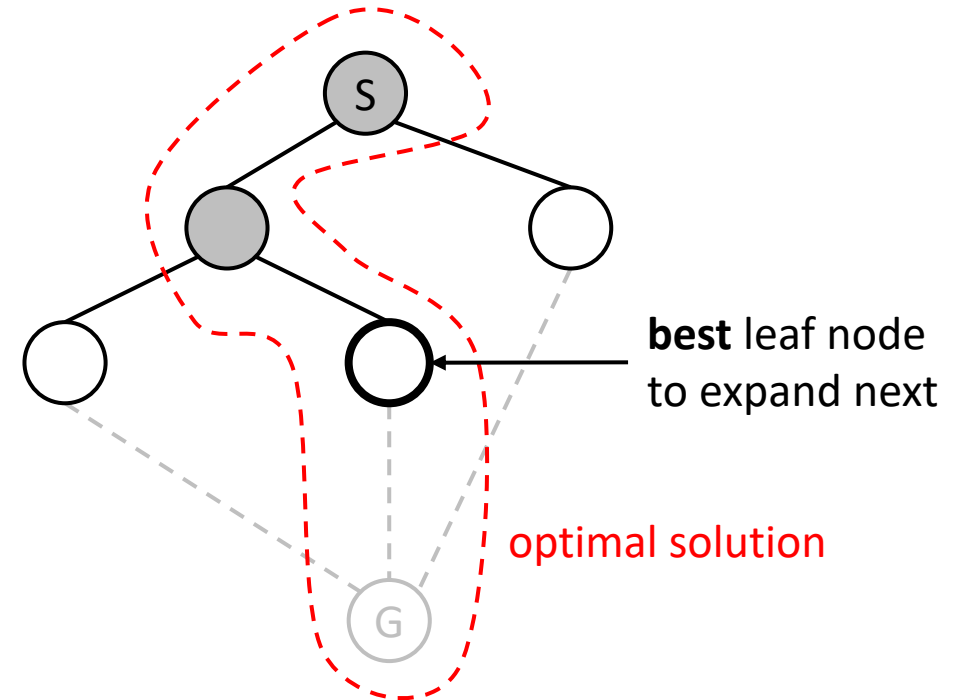
For further implementation hints and examples...

...see the companion [Python notebook](#)

Informed search

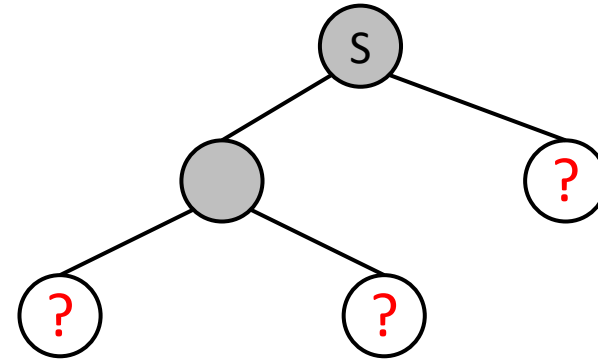
Choosing the *best* node to expand next

- ▶ At each step, (at least) one leaf node is in the path to the **optimal** solution, i.e., a **minimum-cost** path from S to G
- ▶ This would be the **best** node to expand next



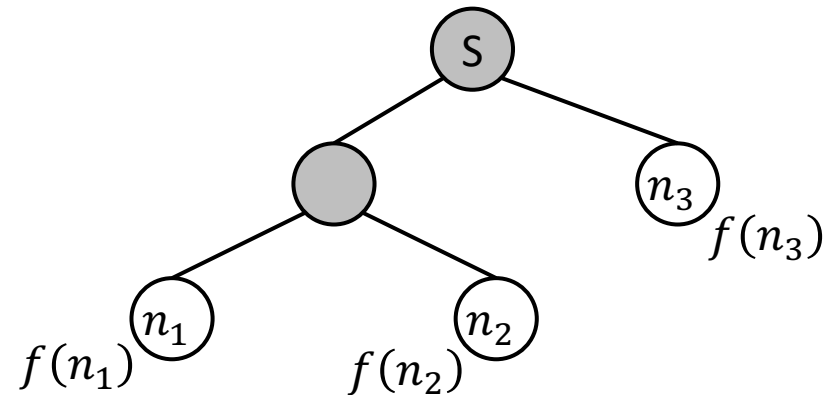
Which is the *best* node?

- ▶ In practice we do not know **which** node it is...
- ▶ ...we will discover it only with hindsight, **after** the optimal solution is found...



Best-first search

- ▶ **Practical** solution: **estimating** the best node to expand next
- ▶ General approach: **best-first search** family of algorithms
 - computing an **evaluation function** $f(n)$ for each leaf node n (by convention, the lower the better)
 - expanding at each iteration the node with the **lowest** $f(n)$ (ties are randomly broken)
 - specific choices of $f(n)$ correspond to **different** best-first search strategies



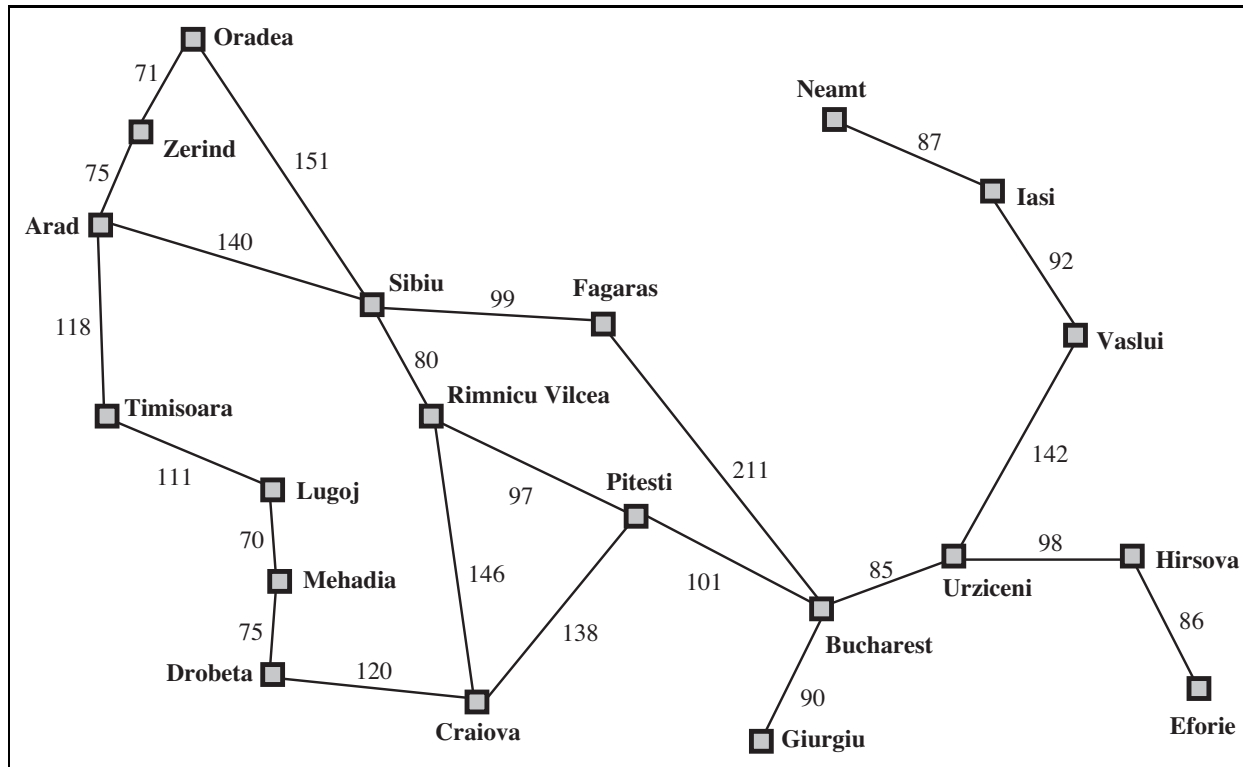
Informed search strategies

- ▶ Problem-specific knowledge (often available in real-world problems) can be exploited to estimate the best node to expand next
- ▶ Usually: an **estimate** of the **lowest path cost** from each state to the goal state
 - the **exact** cost is **unknown** (unless a solution has already been found...), apart from the goal state itself: its exact cost is trivially zero
 - **estimating** it can be **feasible**
- ▶ This kind of best-first strategy is called **informed search**

Heuristic functions

- ▶ For historical reasons a function estimating the lowest path cost from any node n to the goal is called **heuristic** function, denoted by $h(n)$
 - general meaning of *heuristic*: *a rule or piece of information used in or enabling problem-solving or decision-making* (Oxford Dictionary of English)
- ▶ Example: route-finding
 - goal: finding the shortest route from S to D
 - the **exact** road distance from any city to D is unknown beforehand
 - an easy-to-compute estimate: **straight-line distance** (SLD) to D
- ▶ As we will see, **defining "good" heuristics** is crucial for the effectiveness and efficiency of informed search

Example: route-finding



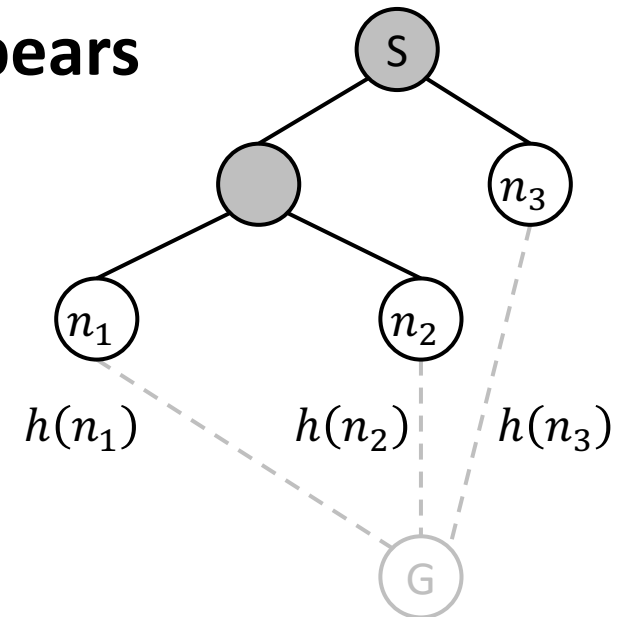
Goal: finding the shortest route from Arad to Bucharest

Heuristic function: SLD to Bucharest

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Greedy best-first search

- ▶ At each step, the heuristic function $h(n)$ provides an estimate of the lowest path cost from each node n to the goal...
- ▶ Greedy search expands the leaf node which **appears closest** to the goal (hence the name "greedy"), i.e., the one with the smallest $h(n)$
- ▶ Evaluation function: $f(n) = h(n)$



Example

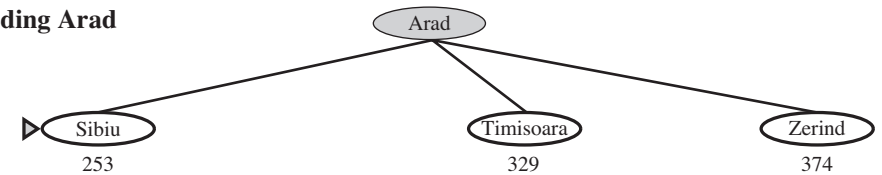
Finding a route from Arad to Bucharest using greedy search with the SLD heuristic

- the value of $f(n) = h(n)$ is shown below each leaf node
- solution found: (Arad, Sibiu, Fagaras, Bucharest) – we already know it is **not** the optimal one...

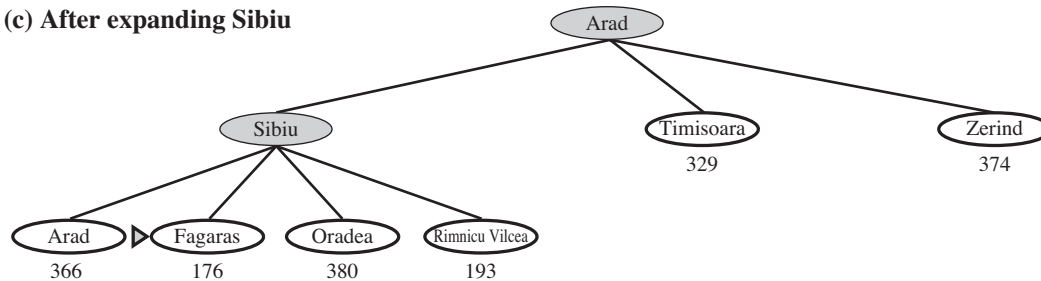
(a) The initial state



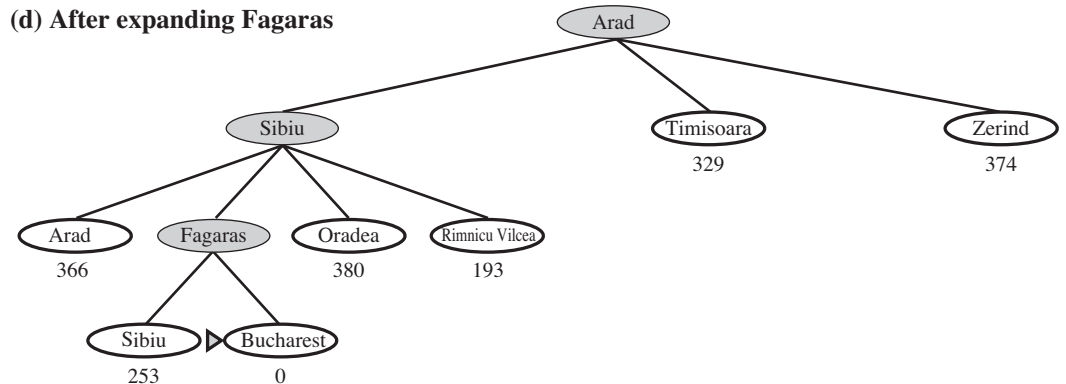
(b) After expanding Arad



(c) After expanding Sibiu



(d) After expanding Fagaras



Properties of greedy search

► Effectiveness

- **complete** (provided that cyclical paths are avoided)
- **not optimal**: a node with lowest **estimated** path cost to the goal is not necessarily on the **real** minimum-cost path...

► Efficiency

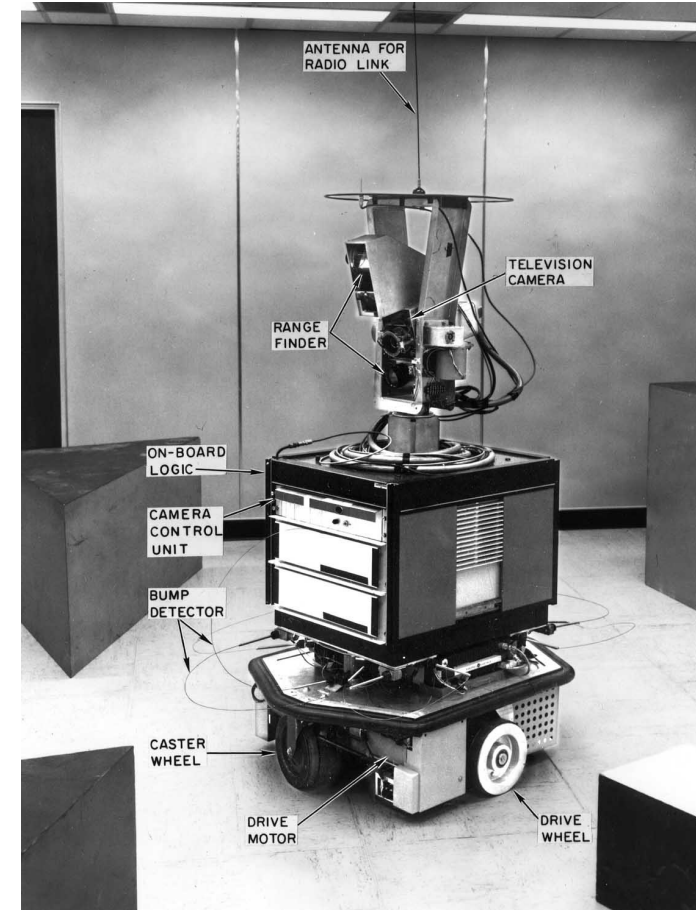
- **time** and **space** complexity: $\mathcal{O}(b^m)$... (m : maximum depth of the search tree)
- however, an **accurate** heuristic can substantially reduce complexity

A* search

- ▶ The most relevant, basic best-first search strategy
- ▶ Developed in the late 1960's at SRI for **robot navigation** tasks
- ▶ A* reads "A star"

Shakey the Robot

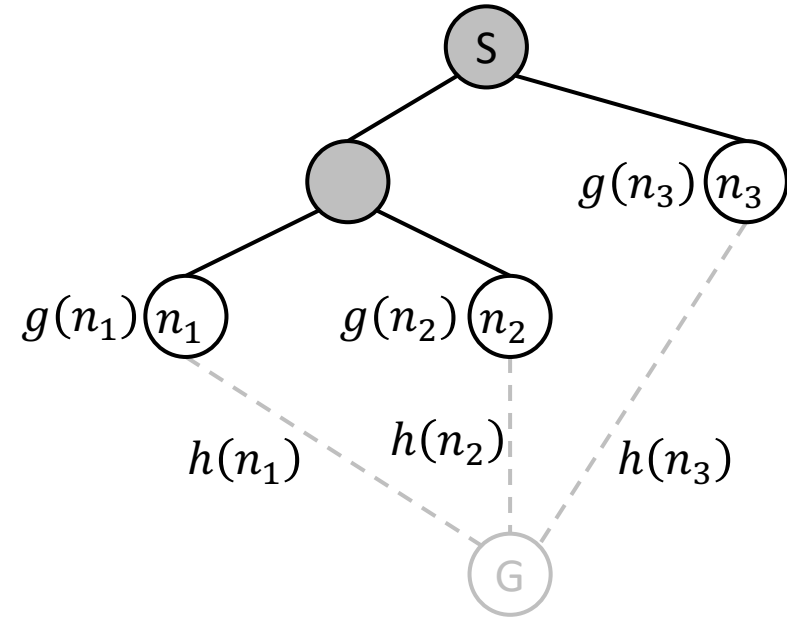
Stanford Research Institute, 1966 – 1972



Rationale of A* search

A **more accurate** estimate of the best node to expand next can be computed taking into account that, for each leaf node n , one knows

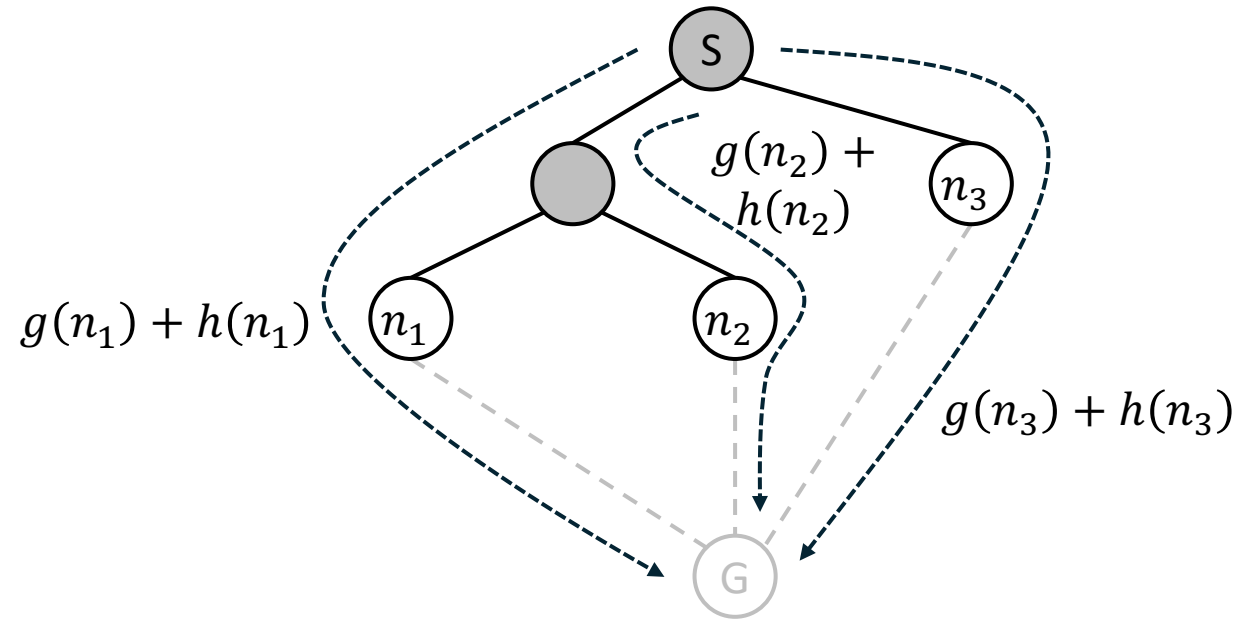
- the **exact** path cost from S to n , denoted by $g(n)$
- the **estimated**, minimum path cost from n to G , $h(n)$



A* search criterion

- ▶ One can therefore estimate the **minimum path cost** from S to G through any node n as $g(n) + h(n)$
- ▶ This leads to the A* criterion: expanding the leaf node with the **minimum estimated solution cost** through it:

$$f(n) = g(n) + h(n)$$



A* search: an example

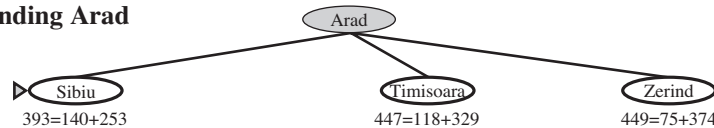
Finding the shortest route from Arad to Bucharest, using the SLD heuristic

- the value of $f(n) = g(n) + h(n)$ is shown below each leaf node
- solution found: (Arad, Sibiu, R. Vilcea, Pitesti, Bucharest) – it is the **optimal** solution: just by chance?

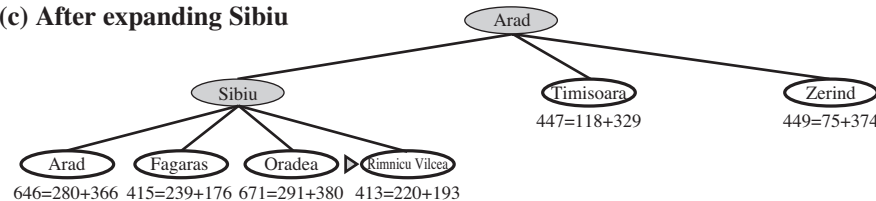
(a) The initial state



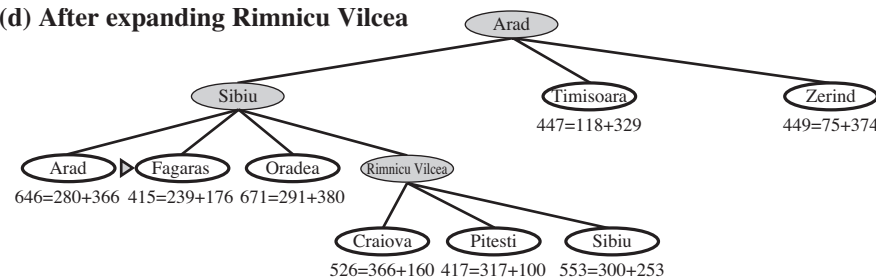
(b) After expanding Arad



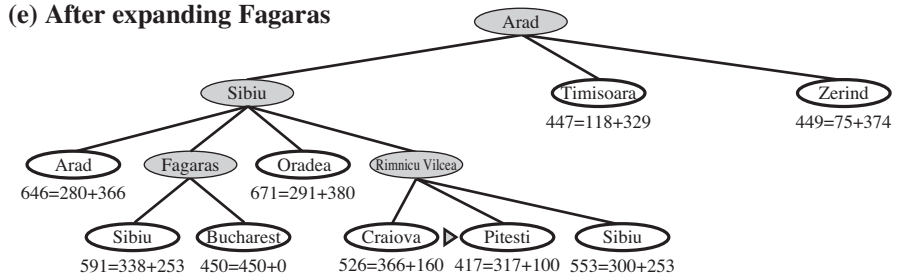
(c) After expanding Sibiu



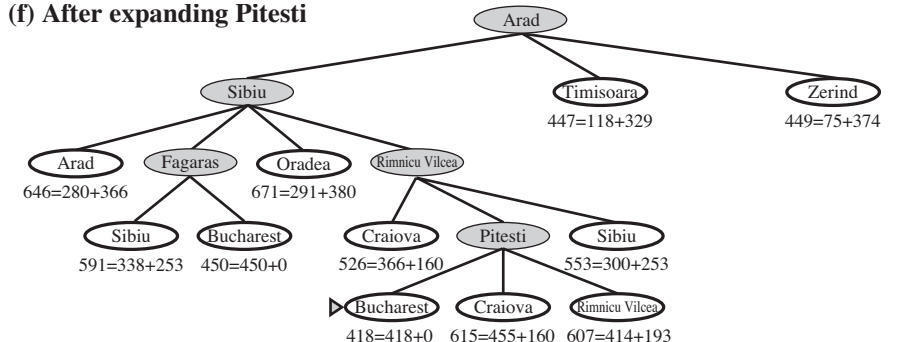
(d) After expanding Rimnicu Vilcea



(e) After expanding Fagaras



(f) After expanding Pitesti



A*: implementation example

See the companion [Python notebook](#)

Admissible heuristics

- ▶ It is not by a lucky chance that A* with the SLD heuristic finds the **optimal** solution to the Arad – Bucharest problem
- ▶ This is because the SLD never overestimates the distance between any pair of cities
- ▶ Heuristic functions that **never overestimate** the cost to reach the goal are said to be **admissible**

Properties of A* search

► Effectiveness

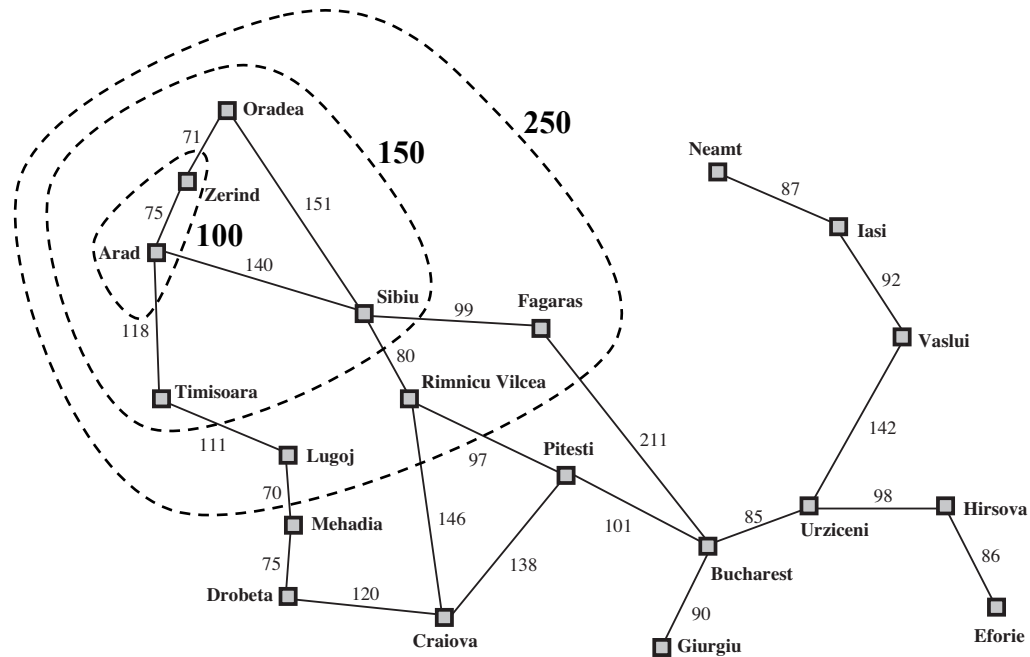
- A* is **complete**
- it is also **optimal**, provided that the heuristic is **admissible** (the proof is left an **exercise** – see the course textbook)

► Efficiency

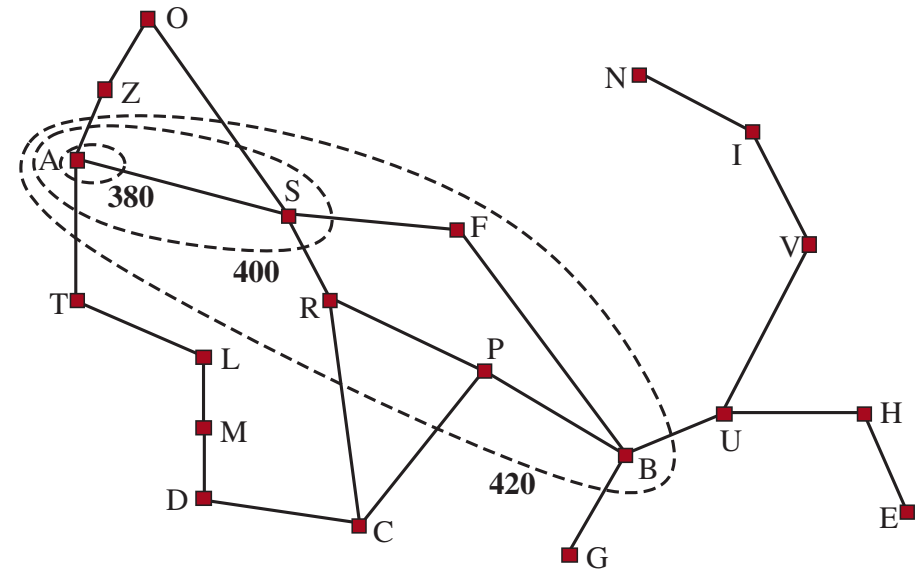
- apparently not better than uninformed search: **time** and **space** complexity are **exponential** in the shallowest solution depth m , $\mathcal{O}(b^m)$...
- however A* turns out to be **optimally efficient**: for any admissible heuristic, it expands the **lowest** number of nodes among algorithms that extend search paths from the root
- in practice it often generates a (much) smaller number of nodes than other uninformed and informed search strategies

A hint on A* efficiency

Uniform-cost search (uninformed): contours at constant path cost $g(n)$ around the initial state



A* search: contours at constant $g(n) + h(n)$ around the initial state: the search is more quickly driven toward the goal state



Defining heuristic functions

Example

- ▶ Defining admissible heuristics is not always straightforward...
 - it is still a research topic for several search problems
- ▶ How could you define one for 8-puzzle?

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

Relaxing problem constraints

- ▶ A general approach
 - **relaxing** some of the **constraints** of the original problem
 - computing the **exact**, minimum path cost from any state S to the goal state (this can be **much easier** than in the original problem)
 - use this value as the heuristic for the original problem (it is **guaranteed** to be **admissible**, since relaxing constraints does not increase the solution cost)
- ▶ An example: route-finding
 - constraints: (1) moving only **along roads** and (2) only to **adjacent** cities
 - relaxing **both** constraints, one could move **straight** to **any** city
 - the corresponding **exact** distance between any pair of cities is the **SLD**

Example: 8-puzzle

- ▶ How to apply the above approach to 8-puzzle?
- ▶ There are two constraints: tiles can be moved only to
 - (1) **adjacent** positions
 - (2) **empty** positions

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

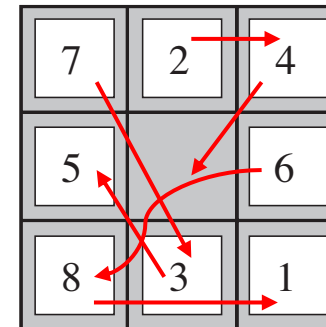
Goal State

Example: 8-puzzle

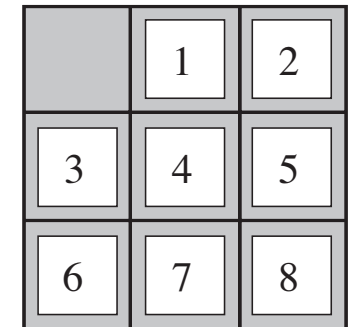
Relaxing **both** constraints

- each tile can be moved to **any** position, even if **not empty**
- the minimum number of moves to the goal state equals the **number of misplaced tiles**
- the corresponding **admissible** heuristic for the original problem:

$$h_1(n) = \text{number of misplaced tiles in } n$$



Start State



Goal State

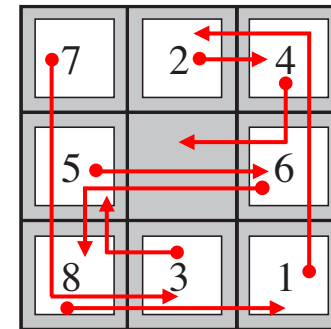
$$h_1(\text{Start State}) = 8$$

Example: 8-puzzle

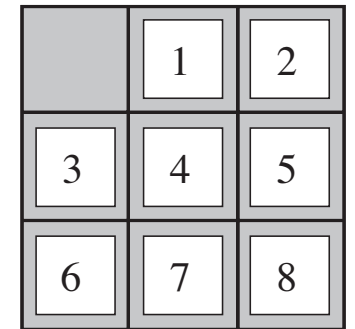
Relaxing **only** constraint (2)

- each tile can be moved to **any adjacent** position, even if **not empty**
- the minimum number of moves to the goal state is the sum of the **Manhattan distances** of each tile to its desired position
- the corresponding **admissible** heuristic:

$$h_2(n) = \sum_{i=1}^8 \text{Manhattan_distance}(\text{tile}_i)$$



Start State



Goal State

$$\begin{aligned} h_2(\text{Start State}) \\ &= 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18 \end{aligned}$$

Choosing among heuristic functions

- ▶ In some cases, like 8-puzzle, more than one heuristic is available
 - are they all equally "good"?
 - how to compare their "quality"?
 - how to choose the "best" one?
- ▶ Consider the two heuristics defined for 8-puzzle
 - $h_1(n)$ = number of misplaced tiles in n
 - $h_2(n) = \sum_{i=1}^8 \text{Manhattan_distance}(\text{tile}_i)$
 - it is not difficult to see that $h_2(n) \geq h_1(n)$ for each n : what does this mean?

Dominating heuristic

- ▶ Given m heuristics $h_1(n), \dots, h_m(n)$, h_j **dominates** the others, if

$$\forall n, i \neq j \quad h_j(n) \geq h_i(n)$$

- ▶ If all heuristics are **admissible**, and h_j **dominates** the others:

$$\forall n, i \neq j \quad c^*(n) \geq h_j(n) \geq h_i(n)$$

where $c^*(n)$ denotes the minimum path cost from n to the goal state

- ▶ In other words, the **dominating** heuristic provides the **most accurate** estimate of the minimum solution cost
- ▶ More accurate heuristics can be expected to improve the **efficiency** of informed search, i.e., reducing time and memory requirements

Selecting the best heuristic function

- ▶ A general criterion for choosing among heuristic functions:
*select the **dominating** heuristic (if any)*
- ▶ Example: 8-puzzle
 - h_2 (Manhattan distance) dominates h_1 (number of misplaced tiles)
 - therefore h_2 can be expected to be more efficient than h_1
 - this can be empirically verified, e.g., see the next example

Empirical comparison of heuristics

- ▶ 8-puzzle: h_1 vs h_2 using A*
 - 100 random problem instances for each value of the shallowest solution depth $d=2, 4, \dots, 20$
 - comparison with IDS (uninformed)
- ▶ Main conclusions: as expected
 - A* is more efficient than IDS
 - h_2 is better than h_1

IDS ran out of memory
for $d \geq 14$...

search cost (average n. of expanded nodes)			
d	IDS	A* (h_1)	A* (h_2)
2	10	6	6
4	112	13	12
6	680	20	18
8	6,384	39	25
10	47,127	93	39
12	3,644,035	227	73
14	–	539	113
16	–	1,301	211
18	–	3,056	363
20	–	7,267	676

Combining heuristic functions

- ▶ What if **none** of the available heuristics $h_1(n), \dots, h_m(n)$ dominates the others?
- ▶ One can **define** a new, **dominating** heuristic by **combining** all of them:
*for each node, select the heuristic providing the **highest** value*
- ▶ Formally: $h(n) = \max_{i=1, \dots, m} h_i(n)$

A* variants

- ▶ In complex real-world problems even A* can run out of memory...
- ▶ Practical solution: A* variants for **space-time trade-off**
 - iterative-deepening A*: limit on the f -cost ($g + h$) rather than the depth
 - recursive best-first search
 - memory-bounded A* and simplified memory-bounded A* (may lose completeness or optimality...)